Sparse-Latent Koopman Autoencoders for Multibasin Dynamical Systems

Anonymous Authors

Abstract

Koopman autoencoders learn coordinates in which nonlinear dynamics are advanced by a linear latent transition. Multibasin systems make this goal structurally difficult: local linearizations, and sometimes linearizations valid across individual basins, can exist around individual attracting sets, but different basins generally require incompatible finite-dimensional, state-reconstructing linear descriptions. We study sparse-latent Koopman autoencoders, where the encoder is biased toward thresholded active sets that are smaller than those of dense controls. The central hypothesis is that this active support can make the relevant basin or local linear law inspectable for the current state. We test this hypothesis by separating forecasting from interpretability, asking whether supports agree with basin labels in controlled benchmarks, whether they can route local predictors without oracle basin labels, and whether sparse-latent models remain competitive at long horizons. The evidence supports a qualified view. On states far from basin boundaries, sparse representations often form stable supports whose members come from a single basin, suggesting that the encoder recovers part of the attractor organization. However, exact support identity is not a reliable global invariant; it is sensitive to thresholds, basin boundaries, and equivalent latent coordinate choices. For deployment, fixed-size support routing is more robust than thresholded exact matching, and support-conditioned local predictors often improve on applying one global latent transition everywhere. Long-horizon forecasting results show that sparse-latent models remain competitive beyond controlled multibasin benchmarks, with stronger transition structure becoming most useful at the longest horizons. The main conclusion is therefore not that each basin has one exact support, but that induced latent sparsity can reveal support families that help identify the relevant basin or local predictor.

1 Introduction

Learning accurate and interpretable models of nonlinear dynamical systems remains a central problem in machine learning, scientific computing, and control. One particularly attractive direction is to seek a representation in which the nonlinear dynamics can become locally linear, since linear models support efficient prediction, estimation, and control while remaining amenable to classical solving tools such as linear quadratic regulators (LQR) (Kalman, 1960) and model predictive control (MPC) (Mayne et al., 2000; Grüne and Pannek, 2017). Classical linearization theory gives local topological conjugacies near hyperbolic equilibria (Grobman, 1959; Hartman, 1960), and Koopman eigenfunction theory extends related linearizing coordinates to entire basins of attraction for stable equilibria and periodic orbits (Lan and Mezić, 2013; Kvalheim and Revzen, 2021). So indeed, a bridge between nonlinear and linear dynamics can be found theoretically.

Koopman operator theory provides a natural way to do this. Instead of evolving the state in the nonlinear dynamical system directly, Koopman theory studies the evolution of observables under a linear operator acting on a lifted function space (Koopman, 1931; Koopman and Neumann, 1932; Mezić, 2005; Brunton et al., 2022). This viewpoint has inspired a large line of literature on data-driven approximations of Koopman dynamics, such as DMD and eDMD (Schmid, 2010;

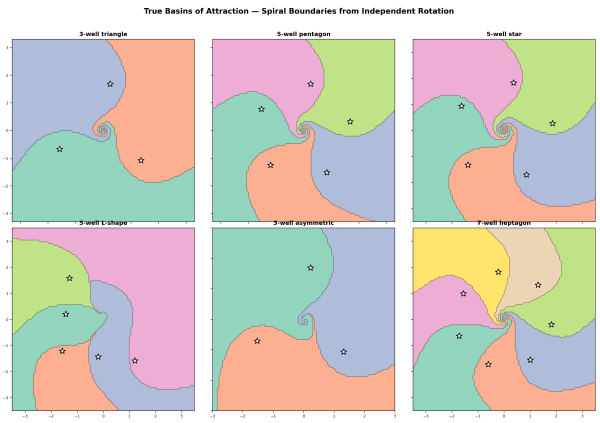
Williams et al., 2015, 2016), and applications of these linear predictors for nonlinear control via MPC (Korda and Mezić, 2018) or LQR (Mamakoukas et al., 2019). More recently, Koopman autoencoders which learn this lift from data have emerged from integrating deep learning with Koopman theory (Takeishi et al., 2017; Lusch et al., 2018; Otto and Rowley, 2019; Azencot et al., 2020; Shi and Meng, 2022; Fathi et al., 2024). Koopman autoencoders learn an encoder which maps a state to a latent representation, a matrix that advances that representation linearly, and a decoder that maps the latent state back to the original state space.

A central challenge of Koopman approximations is that the most interesting nonlinear systems typically contain multiple equilibria or multiple basins of attraction, and theory shows that these systems cannot admit a single finite-dimensional global linearization (Lan and Mezić, 2013; Brunton et al., 2016; Pan and Duraisamy, 2024). For example, a damped mechanical system can settle into different wells; a biological or chemical model can approach different stable equilibria; a controlled system can move between regions with distinct local dynamics. In such multistable systems, it is possible to linearize the dynamics within basins, but the linearizations needed in different basins of attraction are incompatible with each other. Given the theoretical argument, multibasin forecasting with a single global finite-dimensional linearization seems ill-posed; it is more naturally a problem of discovering which local linear description, or which description valid across a basin, applies at the current state.

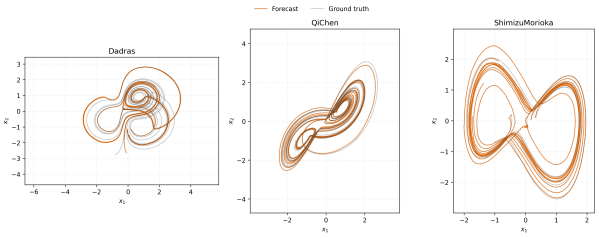
If Koopman models are to serve as a foundation for interpretable multi-regime modeling, then the latent representation of a Koopman autoencoder should do more than forecast well: ideally, it should organize the state space into meaningful local dynamical regimes. Our hypothesis is that sparse coding offers an appealing inductive bias for this purpose. Classical sparse coding and LISTA-style encoders produce piecewise-linear mappings and union-of-subspaces structure in latent space (Olshausen and Field, 1996; Chen et al., 1998; Donoho and Elad, 2003; Gregor and LeCun, 2010). If the latent representation of state produced by a LISTA encoder is sparse, then each state activates only a small subset of latent coordinates. That active subset, which we call a support, is a discrete object that can be inspected and used for routing to correct locally linear dynamics. Then, periodically decoding and re-encoding the state can refresh the quality of the latent embedding over time by re-selecting the correct local dynamics as trajectories move across the state space (Fathi et al., 2024).

This paper tests that hypothesis in a deliberately label-free learning setting. During training and deployment, the model is not told the number of basins, the basin assignment of any trajectory, or which local linear law should be used. Basin labels, when available in controlled benchmarks, are reserved for evaluation. The contribution is an empirical and mechanistic study of whether induced latent sparsity produces support objects that agree with basin labels in evaluation, whether those support objects can route local predictors without oracle labels, and whether the resulting sparse-latent Koopman models remain competitive on long-horizon forecasting tasks.

The paper follows this chain directly. Section 2 formulates multibasin Koopman learning, section 3 explains why sparse supports can identify local dynamical regimes, and section 4 defines the sparse-latent model, support objects, re-encoding rule, and training objective. Section 5 then states the evaluation protocol, including the support-routed prediction diagnostics, and section 6 presents the evidence in the same order as the claim: agreement between supports and basin labels, non-oracle support-routed prediction, support refresh after transfer into a new basin, and long-horizon forecasting.



(a) Representative multibasin benchmark systems.



(b) Long-horizon Dysts forecasting examples at $H20000$.

Figure 1: The paper evaluates two complementary regimes. The procedural multibasin systems provide known basin labels for interpretability diagnostics, while the Dysts systems provide long-horizon forecasting stress tests beyond the controlled low-dimensional setting.

2 Problem formulation

The introduction motivates a multibasin version of Koopman learning. Given sampled trajectories of a nonlinear system, the aim is to learn a state-reconstructing latent representation whose evolution is linear, without being told which basin or local dynamical regime generated each observation. The benchmarks may be defined by continuous-time dynamics, but the learner is only given stored observations. Thus, for each system, we work with the one-step observation map

$$x_{k+1} = F_{\Delta t}(x_k), \tag{1}$$

where $x_k \in \mathbb{R}^{d_x}$ is the state at the kth stored sample, $F_{\Delta t}$ denotes one benchmark observation step, and Δt is the system-specific observation interval. The model does not observe the continuous flow, the vector field, or the internal integration steps used to generate the cached trajectories. Forecasting is therefore defined at the stored sampling cadence: a horizon of H means H applications of $F_{\Delta t}$. Because Δt may differ across benchmark systems, equal values of H need not correspond to equal physical time.

Basins of attraction. Many systems of interest are multistable: different initial conditions can approach different long-run behaviours. A fixed point is a state x^* satisfying $F_{\Delta t}(x^*) = x^*$, equivalently $f(x^*) = 0$ in continuous time. More generally, a trajectory may approach an attractor A, such as a stable equilibrium, limit cycle, torus, or chaotic invariant set. The basin of attraction of A is the set of initial conditions whose trajectories converge to A:

$$\mathcal{B}(A) = \left\{ x_0 \in \mathbb{R}^{d_x} : \operatorname{dist}\left(F_{\Delta t}^k(x_0), A\right) \to 0 \text{ as } k \to \infty \right\}. \tag{2}$$

When multiple attractors coexist, their basins partition the state space up to basin boundaries. This creates the regime of interest for this paper. Within one basin, a useful linear description may exist locally or even basin-wide in appropriate coordinates, but another basin may require an incompatible description. Thus, the task is to learn a representation that can support linear prediction while the appropriate linear law may depend on where the current state lies in the multibasin geometry.

Koopman autoencoders. We study this setting with Koopman autoencoders. The model learns an encoder $E_\theta : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$, a decoder $D_\phi : \mathbb{R}^{d_z} \to \mathbb{R}^{d_x}$, and a latent linear transition matrix $K \in \mathbb{R}^{d_z \times d_z}$. Given an initial observation x_k, the corresponding open-loop h-step forecast is

$$\hat{x}_{k+h} = D_\phi\Big(K^h E_\theta(x_k)\Big). \tag{3}$$

The same matrix K is applied at every step of the rollout, so any state-dependent structure required for prediction must be encoded in the learned representation rather than in a supplied regime label.

Training windows. Training uses windows of adjacent stored observations. For a batch of B windows of rollout length L, $\{x_{b,k}, x_{b,k+1}, \ldots, x_{b,k+L}\}_{b=1}^{B}$, where b indexes windows and ℓ indexes offsets within a window, the model encodes each observed state, rolls out from the initial encoded state, and decodes both reconstructed observations and latent forecasts. The encoded/reconstructed quantities are defined for $\ell = 0, \ldots, L$, while rollout/forecast quantities are defined for $\ell = 1, \ldots, L$:

$$z^{\mathrm{enc}}_{b,k+\ell} = E_\theta(x_{b,k+\ell}), \quad z^{\mathrm{roll}}_{b,k+\ell} = K^\ell z^{\mathrm{enc}}_{b,k}, \quad x^{\mathrm{rec}}_{b,k+\ell} = D_\phi(z^{\mathrm{enc}}_{b,k+\ell}), \quad \hat{x}_{b,k+\ell} = D_\phi(z^{\mathrm{roll}}_{b,k+\ell}). \tag{4}$$

The training losses described later are built from these reconstructed and rolled-out quantities. Each application of K is interpreted as one stored benchmark step, matching the observation map in equation (1).

Model discovers basins unsupervised. In the intended training and deployment setting, the model is not given basin labels, the number of basins, or trajectory-to-basin assignments. When benchmark basin annotations are available, they are reserved for post-hoc evaluation; they are not used to choose the model, train the encoder or transition, or route predictions.

3 Sparse supports as local-regime indicators

The problem formulation leaves a simple constraint: the Koopman autoencoder uses one latent transition K, so any information about which local predictive description is appropriate must be carried by the representation $z = E_\theta(x)$. A dense code may store this information implicitly, but a sparse code exposes this information through its active set $S(x) = \{i : z_i \neq 0\}$. The coefficients $z_{S(x)}$ can encode the continuous state within a local regime, while the support can serve as a discrete, label-free candidate for the current regime. The claim is mechanistic: sparsity gives the model a natural object whose geometric and predictive meaning can be tested.

The following theorem and corollaries explain why supports are the right object to study. Here, "support" means exact zero support, but the thresholded and top-k supports used later are empirical heuristics and not exact algebraic identities. [TODO: Need to review the proofs more thoroughly or get a specialist to do it. (maybe in exchange for authorship?) GPT-5.5 Pro produced the theorems, corollaries, and proofs.]

Theorem 1 (Support-stratified Koopman predictors)**.** *Let $E : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$ be single-valued and piecewise affine on a finite polyhedral stratification. Let $D_\phi(z) = D_{\mathrm{dec}} z + b_{\mathrm{dec}}$ be affine, let $K \in \mathbb{R}^{d_z \times d_z}$, and define $F_h(x) = D_{\mathrm{dec}} K^h E(x) + b_{\mathrm{dec}}$. Then F_h is piecewise affine for every $h \geq 1$. Moreover, after a finite refinement, the signed support of $E(x)$ is constant on each stratum. On any refined stratum $\mathcal{S}$, with exact support $S_{\mathcal{S}} = \mathrm{supp}(E(x))$, we have $F_h(x) = D_{\mathrm{dec}}(K^h)_{:,S_{\mathcal{S}}} E_{S_{\mathcal{S}}}(x) + b_{\mathrm{dec}}$ for all $x \in \mathcal{S}$. For any fixed re-encoding period p, every finite composition of $G_p(x) = D_{\mathrm{dec}} K^p E(x) + b_{\mathrm{dec}}$ is also piecewise affine on a finite itinerary stratification.*

This theorem states that exact supports select active latent coordinates, and hence active columns of K^h, within a finite collection of local affine decoded predictors. The full state-space affine map may still depend on the encoder piece, so two regions with the same support need not have identical local dynamics.

The assumptions of Theorem 1 hold for the sparse encoders used to motivate our model.

Corollary 1 (Exact sparse coding). *Let $D \in \mathbb{R}^{d_x \times d_z}$ be the linear decoder/dictionary and let $\lambda > 0$. Suppose that, for every x, the objective $\frac{1}{2}\|x - Dz\|_2^2 + \lambda\|z\|_1$ has a unique minimizer, and define $E_{\mathrm{sc}}(x)$ to be that minimizer. Then E_{sc} is continuous and piecewise affine on a finite polyhedral stratification. Hence the decoded Koopman predictor $x \mapsto DK^h E_{\mathrm{sc}}(x)$ is support-stratified piecewise affine as in Theorem 1.*

Corollary 2 (Finite-depth LISTA). *Let $T_\tau(y)_i = \mathrm{sign}(y_i)\max\{|y_i| - \tau_i, 0\}$ be coordinatewise soft-thresholding. If $c : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$ is piecewise affine and $u^{(0)}(x) = T_\tau(c(x))$, $u^{(q+1)}(x) = T_\tau(S_{\mathrm{L}} u^{(q)}(x) + c(x))$, then the finite-depth encoder $E_{\mathrm{LISTA}}(x) = u^{(Q)}(x)$ is single-valued and piecewise affine on a finite polyhedral stratification. Hence, Theorem 1 applies. This includes affine, ReLU, and leaky-ReLU pre-codes $c = f_\theta(x)$.*

Formal definitions, detailed statements, and proofs of Theorem 1 and Corollaries 1–2 are deferred to Appendix B.1. Now that we have established the motivation for studying supports, we proceed to an empirical study of (1) whether supports agree with withheld basin labels, and (2) whether those same supports select useful local predictors in practice.

4 Method

4.1 Model class

Can a Koopman autoencoder trained without basin supervision learn a discrete, inspectable object that identifies the relevant basin or local predictive regime? We investigate the following chain

$$\text{sparse latent code} \implies \text{support agrees with basin labels} \implies \text{support selects local prediction.} \tag{5}$$

The model and objective induce the sparse code. The two arrows are evaluated after training: first, by asking whether the support agrees with basin labels that are withheld during training; second, by asking whether the same support can select useful local predictors without receiving a basin label at test time.

The underlying predictor is a Koopman autoencoder with encoder E_θ, decoder D_ϕ, and latent transition K. Using a linear decoder, the maps are

$$z_t = E_\theta(x_t) \in \mathbb{R}^{d_z}, \qquad D_\phi(z) = Dz, \qquad \hat{z}_{t+1} = Kz_t, \qquad \hat{x}_{t+1} = D_\phi(\hat{z}_{t+1}). \tag{6}$$

An active coordinate can therefore be read as selecting a learned reconstruction direction, while the latent state is advanced by the same linear transition between re-encoding events. Sparsity is a property of the learned representation; it is not an assumption that the physical state or the underlying dynamical system is sparse.

Sparse encoder. Our main sparse encoder uses LISTA-style amortized sparse inference (Gregor and LeCun, 2010). The encoder first computes a dense pre-code $c_t = f_\theta(x_t)$ and then applies learned shrinkage refinements, for $q = 0, \ldots, Q-1$:

$$u_t^{(0)} = \mathrm{shrink}(c_t, \tau), \qquad u_t^{(q+1)} = \mathrm{shrink}(Su_t^{(q)} + c_t, \tau), \qquad z_t = u_t^{(Q)}. \tag{7}$$

Here $\mathrm{shrink}(a, \tau) = \mathrm{sign}(a)\max(|a| - \tau, 0)$ is the standard elementwise soft-thresholding map, S is learned, and τ sets the shrinkage scale. The thresholding operation gives the encoder an explicit mechanism for selecting active coordinates, while the learned pre-code and refinement weights allow the selection rule to adapt to the prediction objective.

Transition families. We vary the structure of K separately from the encoder, which separates the effect of induced sparsity from the effect of the latent transition. The dense transition leaves K unrestricted. The block-diagonal transition partitions the latent coordinates into M groups and constrains

$$K_{\text{block}} = \mathrm{blkdiag}(K_1, \ldots, K_M), \tag{8}$$

so each group evolves internally. The soft-block transition keeps K dense but penalizes entries that couple different groups,

$$\mathcal{L}_{\text{offblock}} = \sum_{i,j:\, g(i) \neq g(j)} K_{ij}^2, \tag{9}$$

where $g(i)$ denotes the group containing coordinate i. These groups are architectural coordinates, not basin labels. All basin comparisons are performed only after training.

4.2 Support objects

The central object in the paper is the active set induced by an encoded state. Given $z = E_\theta(x)$, a support rule converts z into a set of active coordinates. The main displays use two support rules, each with a different role:

- **Absolute-threshold support.** $S_{\text{abs}}(x) = \{i : \ |z_i| > 10^{-3}\}$. This is the strictest test of whether a single active set dominates far from basin boundaries.
- **Fixed-size top-8 support.** $S_{\text{top8}}(x) = \text{top}_8(|z|)$, the indices of the eight largest-magnitude coordinates. This is the most deployment-like support because every state receives a route key of the same size.

A third scale-relative rule, $S_{\text{rel}}(x) = \{i : \ |z_i| > 0.1 \max_j |z_j|\}$, is used only in appendix sensitivity diagnostics for centered local laws.

We distinguish exact supports from support families. An *exact support* is the set returned by a support rule. It is a stringent object: if one exact support identifies one basin, the claim is easy to inspect. A *support family* groups nearby exact supports by Jaccard overlap, using threshold 0.5 against the family prototype. This allows one basin or local regime to be represented by several closely related masks, which is expected near basin boundaries or when a weak coordinate crosses an activity threshold. Family-routed prediction assigns a test-time exact support to a previously fitted family only if it matches closely enough; otherwise the prediction falls back to the global transition.

4.3 Periodic re-encoding

If a support indicates the currently relevant local dynamics, then a long forecast should be allowed to update the support as the predicted state moves. We therefore evaluate periodic re-encoding, following Fathi et al. (2024). Starting from $z_k = E_\theta(x_k)$ and a re-encoding period m, the model advances for m Koopman steps, decodes the predicted state, and encodes that prediction again:

$$\tilde{z}_{k+m} = K^m z_k, \qquad \tilde{x}_{k+m} = D_\phi(\tilde{z}_{k+m}), \qquad z_{k+m} = E_\theta(\tilde{x}_{k+m}). \tag{10}$$

The same procedure is repeated over the rollout horizon. Each rollout uses only its own predicted state; it does not use the true future state, a basin label, or an oracle for transitions between basins.

4.4 Support-conditioned rollouts

We also define two support-conditioned one-step rules used in the routing diagnostics. Let c be the support object inferred from the current predicted latent state, let m_c be its binary mask, and let $\bar{z}_c$ be the mean latent state associated with that support object on a disjoint fitting set. The first rule masks the centered input before applying the learned global transition:

$$\hat{z}_{t+1} = \bar{z}_c + K\big(m_c \odot (z_t - \bar{z}_c)\big). \tag{11}$$

The second rule fits a centered local linear predictor for each support object:

$$A_c = \arg\min_A \sum_{t \in \mathcal{I}_c} \|(z_{t+1} - \bar{z}_c) - A(z_t - \bar{z}_c)\|_2^2 + \eta \|A\|_F^2 , \tag{12}$$

and predicts $\bar{z}_c + A_c(z_t - \bar{z}_c)$. These rules test whether the learned support can replace a supplied regime label when selecting a local predictor.

4.5 Training objective

Training uses only adjacent observation windows. For a batch of B windows with prediction horizon L, the model produces reconstructed future states $x^{\text{rec}}_{b,k+\ell}$, latent rollouts $z^{\text{roll}}_{b,k+\ell}$, encoded future latents $z^{\text{enc}}_{b,k+\ell}$, and decoded rollout predictions $\hat{x}_{b,k+\ell}$ as defined in equation (4). The rollout is seeded by the initial state, and the following sums are over offsets $\ell = 1, \dots, L$:

$$\bar{\mathcal{L}}_{\text{pred}} = \frac{1}{BL\sqrt{d_x}} \sum_{b,\ell} \|\hat{x}_{b,k+\ell} - x_{b,k+\ell}\|_2 , \qquad \bar{\mathcal{L}}_{\text{rec}} = \frac{1}{BL\sqrt{d_x}} \sum_{b,\ell} \left\|x^{\text{rec}}_{b,k+\ell} - x_{b,k+\ell}\right\|_2 , \tag{13}$$

$$\bar{\mathcal{L}}_{\text{lin}} = \frac{1}{BL} \sum_{b,\ell} \left\|z^{\text{roll}}_{b,k+\ell} - z^{\text{enc}}_{b,k+\ell}\right\|_2 , \qquad \bar{\mathcal{L}}_{\text{sp}} = \frac{1}{BL} \sum_{b,\ell} \left\|z^{\text{roll}}_{b,k+\ell}\right\|_1 . \tag{14}$$

The observation-space terms are normalized by $\sqrt{d_x}$ so that their scale is comparable across systems of different state dimension. The latent terms are left in their native scale because the latent dimension is fixed within each comparison.

The total objective is

$$\mathcal{L} = \frac{1}{L}\left(\lambda_{\text{pred}}\bar{\mathcal{L}}_{\text{pred}} + \lambda_{\text{rec}}\bar{\mathcal{L}}_{\text{rec}} + \lambda_{\text{lin}}\bar{\mathcal{L}}_{\text{lin}} + \lambda_{\text{sp}}\bar{\mathcal{L}}_{\text{sp}}\right) + \mathcal{L}_{\text{struct}}. \tag{15}$$

Here $\mathcal{L}_{\text{struct}} = 0$ for the dense and block-diagonal transition variants. For the soft-block transition, $\mathcal{L}_{\text{struct}} = \lambda_{\text{off}}\mathcal{L}_{\text{offblock}}$. The prediction and latent-linearity terms make the representation useful for Koopman rollout, the reconstruction term keeps the code tied to the observed state, and the sparsity term encourages selective activation. No loss term uses basin labels, basin counts, or trajectory-to-basin assignments.

Compared with the minimal Koopman autoencoder objective in equation (25), this rollout objective adds decoded multi-step prediction, latent consistency over stored observation windows, explicit sparsity pressure, and any transition-structure penalty. These additions are needed because the paper is not only asking whether one-step latent features can be made approximately linear; it is asking whether the learned support remains useful for finite-horizon prediction and can be inspected as a label-free local-regime object.

5 Experimental procedure

[TODO: Update this section once all the experiments are finally done. The descriptions are stale.]

The experiments follow the chain in equation (5). We first ask whether sparse supports agree with basin labels that were withheld during training. We then ask whether those supports are functionally used by the predictor, whether they can route local forecasts without an oracle label, whether periodic re-encoding keeps the selected support current after basin transfer, and finally whether the same sparse-latent models remain competitive in a longer-horizon forecasting regime without basin labels.

Benchmarks and training. The controlled benchmark contains 17 procedurally generated two-dimensional multibasin systems, spanning multiwell potentials, gated local-linear systems, polygonal arrangements with 3–10 attractors, checkerboard potentials, depth-cascade variants, and transition-rich layouts. Basin labels are available only for evaluation: they are not used for training, model selection, support extraction, or routing. All multibasin models are trained from a distribution that deliberately includes many initial conditions near basin boundaries, so the learned representation must handle both unambiguous basin interiors and boundary-adjacent states. We compare the six model rows shown in table 1: three LISTA sparse encoders (two of which have structured latent transitions), two sparse MLP controls with dense or block-diagonal transitions, and a Dense MLP control with no sparsity penalty. All models except the Dense MLP use ReLU activations. The Dense MLP baseline uses tanh activations since ReLU activations naturally induce sparsity in the network. Each model–system cell is trained for 15 random seeds; paired analyses use the common completed seeds. All rows for Tables 1–3 use length-8 training windows and 200,000 optimization steps under the objective in equation (15). We use a subset of 12 chaotic systems from the Dysts benchmark (Gilpin, 2021, 2023) for forecasting only, since it does not supply basin labels, with length-10 training windows and 100,000 optimization steps.

Forecasting and statistics. Forecasting is evaluated on held-out trajectories at the stored observation cadence. On the multibasin benchmark we report raw mean squared error at $H \in \{100, 500, 1000\}$. On Dysts, the main table reports the completed $dt\times30$ sensitivity packet at $H \in \{100, 500, 1000, 1500, 2000, 3000, 4000, 5000\}$; the older $H\leq60000$ packet is retained as a tiny-step stress-test diagnostic rather than pooled with this coarser-step benchmark. The reported forecasting performance is the best value over a fixed periodic re-encoding grid. Point estimates reported are interquartile means over seeds within each system, followed by an interquartile mean across systems; system-by-system performance is reported in [TODO: cite the section of the appendix]. For the Dysts horizon-trend figure, uncertainty bands are fixed-system seed-bootstrap 95% intervals: the 12 systems are held fixed, seeds are resampled with replacement within each system, per-system seed-IQMs are recomputed, and the cross-system IQM is recomputed for each bootstrap replicate. For the main Dysts model-vs-Dense claim, the test unit is the benchmark system: for each candidate row and horizon, we form the per-system ratio between the candidate seed-IQM and the Dense MLP seed-IQM, test the resulting 12 paired $\log_{10}$ ratios with a one-sided Wilcoxon signed-rank test, and Holm-correct across all Dysts model–horizon comparisons at $\alpha = 0.05$. We also retain a stricter within-system reproducibility count: within each system, paired $\log_{10}$ seed errors are tested against Dense MLP and Holm-corrected across systems; the reported K/N values count systems that pass this correction.

Support diagnostics. [TODO: We might want to check these two paragraphs again based on the new results.] Support–basin agreement is evaluated on held-out states. The main alignment table

uses the absolute-threshold support S_{abs} on the deep-state subset, defined as the top quartile of states by distance from a basin boundary. This focuses the identifiability test on states well inside basins, where a basin-local description should be most stable. We report the mean active-coordinate count $|S_{\text{abs}}|$, basin uncertainty given the exact S_{abs} mask, and basin uncertainty given the corresponding support family F_{abs}. Because S_{abs} is thresholded, $|S_{\text{abs}}|$ can vary by state, basin, system, seed, and latent scaling; it is a compact size diagnostic, not the fixed route size used in the top-8 routing experiments. These quantities must be interpreted together: a low basin uncertainty is meaningful only if the active set is also small enough to be an inspectable object.

We test functional use of the support with a wrong-support ablation. For a held-out deep-state example, the model's inferred support is replaced by a canonical support from another basin and then held fixed for the rollout. The reported ratio is the wrong-support error divided by the ordinary forecast error. Ratios near one indicate that the swap did not matter; large ratios indicate that the predictor depends on the selected support. This ablation is undefined when the deep-state subset contains only one basin, so those systems are excluded from that denominator.

Routing and refresh. The non-oracle routing experiment freezes the trained representation and fits the support-conditioned predictors in equations (11) and (12) on a disjoint fitting set. During evaluation, the route at each step is the model's own predicted top-8 support; if the route has too few fitted transitions, the rollout falls back to the global latent transition. The metric is the routed/global error ratio for the same trained model, so values below one measure the benefit of routing without changing the representation or using a basin label. The routing controls in the appendix replace learned supports with oracle basin partitions, random partitions of matched count, or latent clusters.

The refresh experiment evaluates support selection after controlled transfer from one basin into another. It compares a rollout that keeps using the source-basin support after entry into the target basin with a rollout that periodically decodes, re-encodes, and re-selects its support. Basin labels are used only to construct and score these transfers. The reported quantities are the fraction of post-entry steps routed through the target class and the refreshed/previous forecast-error ratio; because this packet uses one training seed, the paired test unit is the controlled transfer pair rather than the training seed. Additional procedural details are collected in section C.

6 Results

Table 1: Support–basin alignment, support-use ablations, and forecasting on the 17-system benchmark. Values are cross-system IQMs; brackets give within-system Wilcoxon/Holm counts. Lower is better except for wrong-support ratios.

Model	$H(B \mid S_{\text{abs}})\downarrow$	$H(B \mid F_{\text{abs}})\downarrow$	wr. $h{=}1\uparrow$	wr. $h{=}20\uparrow$	$\lvert S_{\text{abs}}\rvert\downarrow$	H100$\downarrow$	H1000$\downarrow$
LISTA-BD	0	**0** [13/13]	$7.38{\times}10^3$ [13/13]	14.9 [11/13]	101 [17/17]	**0.0170** [15/17]	0.0594 [15/17]
LISTA-SB	0	**0** [13/13]	$\mathbf{1.10{\times}10^4}$ [13/13]	14.0 [11/13]	**99.0** [17/17]	0.0198 [15/17]	0.0757 [15/17]
Sparse MLP, BD	0	0.0084 [13/14]	$6.86{\times}10^3$ [13/13]	27.9 [11/13]	107 [17/17]	0.0203 [15/17]	0.0553 [17/17]
Sparse MLP	0	0.00036 [13/13]	$6.53{\times}10^3$ [13/13]	**30.3** [12/13]	107 [17/17]	0.0218 [15/17]	**0.0537** [17/17]
Dense MLP, no shrink	0.0021	0.900 *[baseline]*	8.25 *[baseline]*	1.37 *[baseline]*	254 *[baseline]*	0.762 *[baseline]*	3.27 *[baseline]*

Thresholded supports identify basin membership on deep states (table 1). On the deep-state slice, the absolute-threshold support is basin-pure for the sparse rows: every sparse model has $H(B \mid S_{\text{abs}}) = 0$. The dense MLP also has low exact-mask uncertainty, but only with

a nearly fully active mask ($|S_{\text{abs}}| = 254$), so its exact support is not an inspectable sparse object. The support-family view separates the rows more clearly: the sparse models have almost zero $H(B \mid F_{\text{abs}})$, the LISTA rows have exactly zero, and the dense baseline has high family-level uncertainty. The size column should be read as an averaged thresholded active-coordinate count: the sparse rows are far smaller than the dense baseline, but they are not hard 8-sparse under S_{abs}. Thus, the evidence is not that each basin has one tiny exact mask; it is that sparse encoders expose thresholded masks and low-uncertainty support families that identify basin membership on states far from basin boundaries. The per-(system, seed) distributions in figure 2 confirm that the gap is consistent rather than driven by outliers.

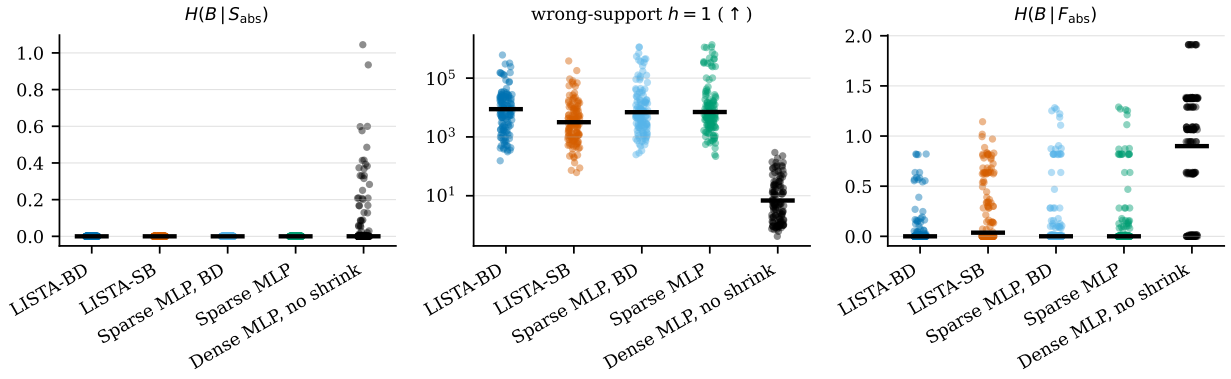


Figure 2: Per-seed support diagnostics on the deep-state slice. Each dot is one system–seed pair and the black bar is the cross-system IQM. Lower is better for the entropy columns; higher is better for the wrong-support ratio.

Accurate forecasts require same-basin supports (table 1). Support–basin agreement alone does not imply that the support affects prediction. The wrong-support ablation in table 1 constructs a canonical S_{abs} mask for each basin, then rolls out after freezing the latent state to a canonical mask from a different basin. This is an intervention on the active coordinates. Under this intervention, every sparse model degrades by several orders of magnitude at one-step horizon and remains markedly worse twenty steps out, while the dense MLP baseline is much less affected. This shows that the basin-aligned active coordinates are functionally coupled to prediction on the testable deep-state systems. It does not by itself show that S_{abs} alone is sufficient for forecasting, or that the ordinary rollout explicitly routes by this thresholded support.

Support families select local predictors without basin labels (table 2). Table 2 asks whether the support can be used after training as a decision rule, not only whether prediction depends on the support. Support-local prediction fits one local linear map for each exact top-8 support, while family-local prediction first groups nearby supports and fits one map per support family. At each rollout step, the model selects the predictor from its own predicted support, and the comparison is always against the same model using its global latent transition. The main positive result appears when nearby supports are grouped into families: these groups improve the LISTA rollouts, including on the deep-state slice in table 16, whereas exact top-8 support identity is less stable. The partition controls in table 7 calibrate the claim. Oracle basin partitions test whether useful local predictors exist when the benchmark partition is supplied; support families test whether the learned sparse representation provides a label-free substitute for that unavailable oracle; random

matched partitions and latent clusters test whether improvement follows merely from subdividing the data. Thus the evidence is not just that supports affect prediction, but that learned support families can serve as label-free selectors for local predictors.

Table 2: All-state support-routed forecasting with top-8 routes. Entries are routed/global MSE-ratio IQMs with within-system Wilcoxon/Holm counts in brackets. LISTA-SB uses the matched d_z=256 control; lower is better.

	$H100$			$H1000$		
Model	Gated	Support-local	Family-local	Gated	Support-local	Family-local
LISTA-SB	0.838 [1/17]	0.825 [2/17]	0.204 [11/17]	**0.778** [8/17]	**0.757** [8/17]	$\mathbf{7.31\times10^{-4}}$ [15/17]
LISTA-BD	0.919 [1/17]	0.905 [2/17]	**0.148** [13/17]	0.846 [8/17]	0.847 [8/17]	0.0132 [15/17]
Sparse MLP, BD	**0.826** [0/17]	0.762 [1/17]	0.271 [7/17]	0.946 [2/17]	0.915 [2/17]	131 [17/17]
Sparse MLP	0.858 [0/17]	**0.756** [1/17]	0.673 [6/17]	0.945 [1/17]	0.877 [1/17]	1.61×10^{3} [17/17]
Dense MLP	0.981 [0/17]	0.993 [0/17]	1.04×10^{3} [0/17]	0.886 [0/17]	0.970 [0/17]	675 [1/17]

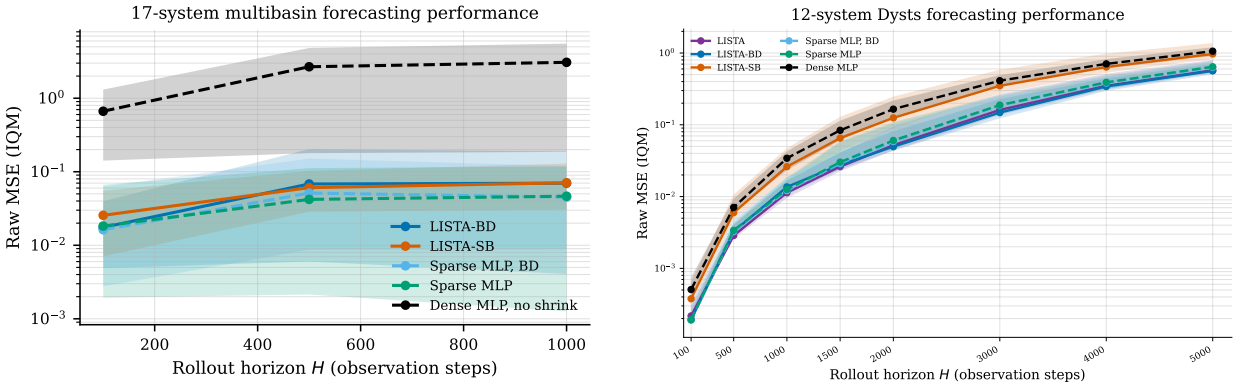
Refreshing the support with periodic re-encoding tracks basin transfer. The routing test starts inside a single basin. For deployment over long horizons, the support must remain current as a trajectory crosses basin boundaries. Table 3 compares a stale source-basin support held throughout the rollout against a support refreshed by the periodic re-encoding rule of equation (10) after a controlled entry into a target basin. Refreshed rollouts route through the target basin on most post-entry steps and reduce forecast error by roughly one order of magnitude at period 1 and more strongly at period 10, with per-system tests passing on all 12 transfer-covered systems for both LISTA variants and both re-encoding periods. The improvement is stable across the re-encoding periods we evaluated, indicating that the mechanism is not sensitive to a particular schedule. Periodic re-encoding therefore supplies a label-free way of keeping the local description aligned with the current region of state space, completing the deployment-time picture begun by the routing test.

Table 3: Support refresh after controlled basin transfer. Route-target is the fraction of post-entry steps routed to the target basin; the MSE ratio compares refreshed support with stale source support; $K/12$ gives Wilcoxon/Holm counts. LISTA-SB uses the matched d_z=256 control.

Model	Period	Route-target	Fallback	MSE ratio vs prev. support: IQM $[Q25, Q75]$	$K/12$
LISTA-SB	1	**0.95** (0.32)	**0.052**	**0.107** $[\mathbf{0.085}, \mathbf{0.142}]$	**12/12**
LISTA-SB	10	0.92 (0.24)	0.085	**0.031** $[\mathbf{0.027}, \mathbf{0.035}]$	**12/12**
LISTA-BD	1	0.94 (0.26)	0.058	0.136 $[0.088, 0.200]$	**12/12**
LISTA-BD	10	**0.93** (0.24)	**0.073**	0.038 $[0.024, 0.046]$	**12/12**

Sparse models improve controlled multibasin forecasting, while Dysts is a mixed stress test. The interpretability gains come at no forecasting cost on the controlled benchmark. Every sparse model reduces cross-system MSE by roughly two orders of magnitude relative to the dense baseline at the longest evaluated horizon, with per-system improvements on a strong majority of the 17 systems across horizons from 100 to 1000 stored steps (table 1, figures 3a and 4); the dense baseline carries a substantially heavier upper tail at every horizon. The two sparse-MLP controls

track the LISTA variants closely, indicating that the active ingredient is induced latent sparsity rather than the LISTA architecture per se. On the 12 chaotic Dysts systems, the $dt\times30$ sensitivity packet is fully finite in every model row and horizon (tables 4 and 5 and figure 3b). LISTA is best at $H500$–$H1500$, LISTA-BD is best from $H2000$ through $H5000$, and Sparse MLP-BD is best at $H100$. Relative to the Dense MLP baseline, LISTA improves the per-system seed-IQM on all 12 systems from $H500$ onward and is significant at every horizon under the aggregate system-level Wilcoxon/Holm test; LISTA-BD is significant from $H1000$ through $H5000$ under the same aggregate test. The within-system Holm counts are lower because they ask a different, stricter question: how many individual systems show seed-level reproducible improvement after correction across systems. Under that reproducibility count, LISTA clears 5–6 systems from $H500$ onward and LISTA-BD clears 3–5 systems at the longest horizons. The sparse MLP controls also improve most systems, while LISTA-SB has lower aggregate error than Dense MLP at most horizons but does not survive all-comparison Holm correction in the aggregate test. Thus the coarser-step Dysts result supports forecasting competitiveness and a discretization-sensitive interpretation of the older block-diagonal failure, but it is not evidence of a LISTA-specific advantage independent of induced sparsity. Because Dysts trajectories carry no basin labels, this evidence speaks only to forecasting and does not bear on support–basin alignment.



(a) Fixed-17 multibasin benchmark.

(b) 12-system Dysts $dt\times30$ benchmark.

Figure 3: Forecasting horizon trends. In the fixed-17 panel, lines are cross-system IQMs of per-system IQM-over-seeds and bands show the 25–75 system percentile range. In the Dysts panel, lines are cross-system IQMs of per-system seed-IQMs and bands show fixed-system seed-bootstrap 95% intervals. Raw MSE is shown on a log scale in the Dysts panel because the displayed IQMs span roughly 5.6×10^3 from the shortest to longest horizon.

7 Related work

Koopman learning. Koopman operator theory represents nonlinear dynamics through a linear operator on observables (Koopman, 1931; Mezić, 2005). EDMD approximates this operator with a chosen dictionary (Williams et al., 2015). Neural Koopman methods learn the dictionary or invariant subspace from data (Takeishi et al., 2017; Lusch et al., 2018; Otto and Rowley, 2019). Consistent and course-corrected Koopman autoencoders improve rollout behavior through additional losses and correction mechanisms (Azencot et al., 2020; Fathi et al., 2024). Our contribution is not a new claim that neural Koopman autoencoders can forecast. It is an empirical study of whether sparse latent supports carry information about the relevant basin or local dynamics.

Table 4: Dysts $dt\times30$ forecasting on the 12-system shortlist. Each cell is the cross-system IQM of per-system seed-IQMs, followed by $[K/12]$, the number of systems whose within-system one-sided paired Wilcoxon test on log-MSE deltas against the Dense MLP clears Holm correction at α=0.05. Dense MLP is the baseline. All rows have full collection and median full-horizon finite coverage of 1 at every reported horizon.

Model	H100	H500	H1000	H1500	H2000	H3000	H4000	H5000
LISTA	2.18×10^{-4} [3/12]	**0.0029** [6/12]	**0.011** [5/12]	**0.026** [5/12]	0.052 [5/12]	0.161 [5/12]	0.349 [5/12]	0.574 [5/12]
LISTA-BD	1.94×10^{-4} [2/12]	0.0033 [2/12]	0.014 [4/12]	0.028 [4/12]	**0.05** [4/12]	**0.148** [5/12]	**0.342** [4/12]	**0.564** [3/12]
LISTA-SB	3.78×10^{-4} [0/12]	0.006 [0/12]	0.026 [0/12]	0.065 [0/12]	0.125 [0/12]	0.349 [0/12]	0.639 [0/12]	0.969 [0/12]
Sparse MLP	1.92×10^{-4} [3/12]	0.0034 [4/12]	0.013 [6/12]	0.03 [5/12]	0.061 [2/12]	0.188 [2/12]	0.391 [3/12]	0.643 [2/12]
Sparse MLP-BD	$\mathbf{1.9\times10^{-4}}$ [3/12]	0.0033 [3/12]	0.012 [2/12]	0.029 [7/12]	0.058 [6/12]	0.185 [2/12]	0.381 [3/12]	0.628 [2/12]
Dense MLP	5.08×10^{-4}	0.0071	0.034	0.084	0.165	0.412	0.706	1.06

Table 5: Aggregate Dysts $dt\times30$ model-vs-Dense tests. Each cell is the cross-system IQM of per-system candidate/Dense seed-IQM ratios; lower is better and values below 1 favor the candidate. p_{Holm} is the one-sided Wilcoxon signed-rank p-value over the 12 per-system $\log_{10}$ ratios, Holm-corrected across all candidate-model and horizon comparisons in this table. Bold entries pass $p_{\text{Holm}} < 0.05$.

Model	H100	H500	H1000	H1500	H2000	H3000	H4000	H5000
LISTA	**0.558** ($p_{\text{Holm}}=0.014$)	**0.48** ($p_{\text{Holm}}=0.0098$)	**0.416** ($p_{\text{Holm}}=0.0098$)	**0.406** ($p_{\text{Holm}}=0.0098$)	**0.419** ($p_{\text{Holm}}=0.0098$)	**0.486** ($p_{\text{Holm}}=0.0098$)	**0.559** ($p_{\text{Holm}}=0.0098$)	**0.595** ($p_{\text{Holm}}=0.0098$)
LISTA-BD	0.522 ($p_{\text{Holm}}=0.148$)	0.516 ($p_{\text{Holm}}=0.154$)	**0.453** ($p_{\text{Holm}}=0.042$)	**0.439** ($p_{\text{Holm}}=0.017$)	**0.438** ($p_{\text{Holm}}=0.0098$)	**0.514** ($p_{\text{Holm}}=0.014$)	**0.581** ($p_{\text{Holm}}=0.026$)	**0.607** ($p_{\text{Holm}}=0.042$)
LISTA-SB	0.881 ($p_{\text{Holm}}=0.509$)	0.921 ($p_{\text{Holm}}=0.509$)	0.783 ($p_{\text{Holm}}=0.323$)	0.78 ($p_{\text{Holm}}=0.154$)	0.739 ($p_{\text{Holm}}=0.154$)	0.797 ($p_{\text{Holm}}=0.154$)	0.873 ($p_{\text{Holm}}=0.323$)	0.898 ($p_{\text{Holm}}=0.509$)
Sparse MLP	0.514 ($p_{\text{Holm}}=0.148$)	0.511 ($p_{\text{Holm}}=0.105$)	**0.51** ($p_{\text{Holm}}=0.026$)	**0.587** ($p_{\text{Holm}}=0.0098$)	**0.599** ($p_{\text{Holm}}=0.0098$)	**0.617** ($p_{\text{Holm}}=0.014$)	**0.664** ($p_{\text{Holm}}=0.017$)	**0.694** ($p_{\text{Holm}}=0.031$)
Sparse MLP-BD	0.512 ($p_{\text{Holm}}=0.126$)	**0.53** ($p_{\text{Holm}}=0.048$)	**0.517** ($p_{\text{Holm}}=0.014$)	**0.576** ($p_{\text{Holm}}=0.0098$)	**0.57** ($p_{\text{Holm}}=0.0098$)	**0.598** ($p_{\text{Holm}}=0.014$)	**0.669** ($p_{\text{Holm}}=0.026$)	**0.718** ($p_{\text{Holm}}=0.042$)

Multibasin Koopman structure. Multiple invariant sets complicate naive interpretations of a single global finite-dimensional Koopman representation. Prior theory shows that global lifting and reconstruction depend on invariant-set structure, spectra, and symmetry (Lan and Mezić, 2013; Pan and Duraisamy, 2024). This motivates our use of support families and top-k supports rather than a universal exact-support claim.

Sparse coding and LISTA. Sparse coding is a classical representation principle in which signals are reconstructed from a small number of active atoms (Olshausen and Field, 1996). LISTA learns fast amortized sparse inference (Gregor and LeCun, 2010). We use this sparse-inference bias inside a dynamical model and evaluate whether the resulting active atoms form support objects that agree with benchmark basin labels.

Switching and local linear models. Switching state-space models, hybrid systems, and local Koopman methods explicitly introduce modes or local operators (Linderman et al., 2017; Torrisi and Bemporad, 2004; Mamakoukas et al., 2019). Our model differs because it does not train with known basin labels or a known basin count. The switching behavior, when it appears, is implicit in the learned sparse support.

8 Limitations and conclusion

The results support a qualified, falsifiable claim. Sparse-latent KAEs can learn support objects that agree with basin labels on states far from basin boundaries, and fixed-size top-8 supports can route useful local forecasts without oracle labels. Exact support uniqueness is not guaranteed globally. It is strongest away from separatrices and should not be used as a brittle thresholded deployment rule.

The evidence also separates sparsity from LISTA-specificity. Sparse MLP controls are strong, so the paper should not claim that LISTA is the only route to the observed behavior. The safer conclusion is that induced sparsity is a useful inductive bias for Koopman representations whose active supports agree with basin labels, with LISTA providing one structured implementation of that bias.

Several limitations remain. The current true-geometry evidence is mixed, so we do not claim recovery of true Jacobian eigendirections. Controlled transfer between basins is positive for dense sparse-latent top-8 supports and for support families, but it is not uniformly positive for every transition structure or every exact support definition. The Dysts evidence is also benchmark-discretization sensitive: under the older tiny-step $H \leq 60000$ stress test, the block-diagonal LISTA row diverged on several chaotic systems, while the $dt \times 30$ packet reported here is fully finite and competitive. The benchmark systems used for support-label comparisons are low-dimensional, and higher-dimensional systems may require additional structure or weaker support-family interpretations.

Overall, sparse support objects provide a useful bridge between forecasting and interpretability. They expose discrete structure related to basin membership and local dynamics inside a Koopman autoencoder, and the current evidence shows that this structure can agree with basin labels and route local predictive laws. The most robust paper claim is not that each basin has one exact support, but that induced sparse support objects can reveal and exploit local dynamical structure when evaluated with appropriate support definitions and falsification diagnostics.

A Additional background

A.1 Koopman operators and finite-dimensional approximations

Consider a discrete-time nonlinear dynamical system

$$x_{t+1} = f(x_t), \qquad x_t \in \mathcal{X} \subset \mathbb{R}^{d_x}. \tag{16}$$

Koopman operator theory studies the evolution of observables rather than the evolution of the state directly. An observable is a measurement function $h : \mathcal{X} \to \mathbb{R}$, and the Koopman operator $\mathcal{K}$ acts by composition:

$$(\mathcal{K}h)(x) = h(f(x)). \tag{17}$$

The map f may be nonlinear, but $\mathcal{K}$ is linear on the function space of observables. If a finite vector of observables

$$\Phi(x) = \big[h_1(x), \ldots, h_{d_z}(x)\big]^\top \tag{18}$$

spans a Koopman-invariant subspace, then there is a matrix $K_\Phi \in \mathbb{R}^{d_z \times d_z}$ such that

$$\Phi(x_{t+1}) = \Phi(f(x_t)) = \mathcal{K}\Phi(x_t) = K_\Phi \Phi(x_t). \tag{19}$$

In that case the nonlinear dynamics are exactly linear in the observable coordinates $z_t = \Phi(x_t)$. Learning Koopman models can therefore be viewed as searching for observables whose span is invariant enough to support useful finite-dimensional linear prediction.

In practice, the infinite-dimensional operator is approximated from data. Given snapshot pairs $\{(x_t, x_{t+1})\}$ and a chosen feature map Φ, dynamic mode decomposition and extended dynamic mode decomposition estimate a finite transition by least squares (Schmid, 2010; Rowley et al., 2009; Tu et al., 2014; Williams et al., 2015, 2016):

$$K_\Phi = \arg\min_{\hat{K}} \sum_t \Big\| \Phi(x_{t+1}) - \hat{K}\Phi(x_t) \Big\|_2^2. \tag{20}$$

DMD is the special case $\Phi(x) = x$, so it fits a linear state-space surrogate without a learned nonlinear lift. EDMD extends this by choosing a richer feature dictionary. Koopman autoencoders replace the fixed dictionary with a learned encoder and decoder, which is the setting used in this paper.

The same lifted-linear viewpoint is also useful for controlled systems, even though the experiments in this paper are autonomous. For dynamics

$$x_{t+1} = f(x_t, u_t), \tag{21}$$

Koopman control models often learn observables $z = \psi(x)$ and matrices A, B, C such that

$$z_{t+1} \approx Az_t + Bu_t, \qquad x_t \approx Cz_t. \tag{22}$$

The resulting predictor can be combined with linear control tools such as LQR or MPC while the nonlinearity is absorbed into the lifting map and decoder (Kalman, 1960; Korda and Mezić, 2018; Mayne et al., 2000). Linear latent dynamics also simplify system identification and long-horizon rollout because the same finite matrix can be fitted, analyzed, and iterated directly. This paper does not evaluate control policies, but this motivation is important for interpretation: if a support can identify the currently relevant local linear law, then it is not merely a post-hoc label, but a possible routing object for prediction, planning, or control.

A.2 Why multibasin systems require local structure

The theoretical obstruction in multibasin systems is not the absence of local linear descriptions. Near appropriate attracting sets, classical linearization and Koopman eigenfunction constructions can produce useful local or basin-restricted coordinates. The obstruction is instead global: a single finite-dimensional, state-reconstructing Koopman-invariant subspace is highly restrictive when multiple invariant sets coexist. In particular, results summarized by Lan and Mezić (2013) and Brunton et al. (2016) show that exact finite-dimensional invariant subspaces containing the state coordinates are compatible only with limited global attractor structure.

Equivalently, an attractor A induces a basin

$$\mathcal{B}(A) = \{x \in \mathcal{X} : \mathrm{dist}(f^t(x), A) \to 0 \text{ as } t \to \infty\}, \tag{23}$$

and different basins can require different local coordinates or different linear predictive laws. Basin boundaries are therefore precisely the regions where a representation that was valid for one local chart may need to switch to another.

For the present paper, this motivates a cautious formulation. A learned finite-dimensional Koopman representation on a multibasin system should not be expected to be one exact global linearization that reconstructs every basin simultaneously. It must either approximate across incompatible regions, or it must contain some implicit information about which local description is active. Sparse supports are useful because they make one such piece of information discrete and inspectable: the support can identify a basin, basin part, or local predictive chart while the coefficient values carry within-chart state information.

A.3 Koopman autoencoders and periodic re-encoding

A standard Koopman autoencoder learns an encoder $E_\theta : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$, decoder $D_\phi : \mathbb{R}^{d_z} \to \mathbb{R}^{d_x}$, and latent transition K so that

$$z_t = E_\theta(x_t), \qquad z_{t+1} \approx Kz_t, \qquad x_t \approx D_\phi(z_t). \tag{24}$$

A minimal one-step objective has the form

$$\min_{E_\theta, D_\phi, K} \sum_t \|x_t - D_\phi(E_\theta(x_t))\|_2^2 + \lambda_{\text{lin}} \|E_\theta(x_{t+1}) - K E_\theta(x_t)\|_2^2 . \tag{25}$$

This objective is useful as a reference point, but it is not sufficient for the present study. It does not directly train decoded multi-step rollouts, does not impose a sparse support object, and does not test whether the support itself selects a useful local predictive rule. The training objective in section 4.5 therefore keeps the autoencoding and latent-linearity terms but adds decoded rollout prediction over windows, sparsity on the rolled latent states, and optional transition-structure penalties.

After training, a one-shot Koopman rollout encodes once, advances by repeated multiplication with K, and decodes at the requested horizon. Periodic re-encoding instead interrupts the latent rollout after a fixed number of stored observation steps. For period r,

$$\tilde{z}_{t+r} = K^r z_t, \qquad \tilde{x}_{t+r} = D_\phi(\tilde{z}_{t+r}), \qquad z_{t+r} = E_\theta(\tilde{x}_{t+r}). \tag{26}$$

This is the same mechanism defined in equation (10). It uses only the model's own predicted state and is therefore not a teacher-forced correction. The point is to refresh the latent embedding and, in sparse models, to refresh the support as the predicted state moves through regions where a different local linear description may be more appropriate (Fathi et al., 2024).

A one-shot rollout can fail for two related reasons. First, the learned encoder and decoder are not exact inverses, so repeated multiplication by K can push the latent state away from the region where the decoder was trained to reconstruct accurately. Second, finite-dimensional Koopman invariance is only approximate, especially near separatrices or transitions between locally valid charts. Periodic re-encoding addresses both issues by projecting the model's own decoded prediction back through the encoder, where the sparse thresholding rule can select a support appropriate for the current predicted state rather than preserving a stale support from the initial state.

In multibasin terms, the support selected at the initial state should not be assumed to remain valid after the predicted trajectory enters another basin or another boundary-adjacent chart. Re-encoding is therefore evaluated not as teacher forcing, but as a label-free opportunity for the encoder to refresh both the continuous latent coefficients and the discrete support.

A.4 Sparse coding and LISTA

Classical sparse coding seeks a code z that reconstructs a signal from a small set of active dictionary atoms:

$$\min_z \frac{1}{2} \|x - Dz\|_2^2 + \lambda \|z\|_1 , \tag{27}$$

where D is a dictionary and $\lambda > 0$ controls sparsity (Olshausen and Field, 1996; Chen et al., 1998; Donoho and Elad, 2003). The active set of the solution is piecewise constant over regions of the input space, while the coefficient values vary continuously within a fixed active set. This gives a union-of-subspaces geometry: each support indexes a lower-dimensional chart, and nearby supports can be grouped into a support family when threshold noise or boundary effects fragment the exact masks.

LISTA replaces an iterative sparse-coding solver with a learned finite-depth shrinkage network (Gregor and LeCun, 2010). In the notation of equation (7), the encoder forms a pre-code from the input and repeatedly applies learned affine refinement followed by soft thresholding. The thresholding operation is the important ingredient for this paper: it turns the latent code into both continuous coefficients and an explicit support. The experiments then ask whether that support

agrees with benchmark basin labels, whether it is functionally used by the predictor, and whether it can route local linear laws without a supplied basin label.

Observed supports also define a simple label-free graph: nodes are exact supports or merged support families, and edges are support transitions seen along rollouts. This graph view is not used as a training signal in the reported experiments, but it explains why supports are useful objects to expose. A support node can index a local predictor, a support-family node can absorb small threshold perturbations, and an observed edge can record when the representation moves from one local chart to another. The routing experiments in table 2 test the one-step predictive version of this idea without assuming a known number of basins or known basin membership during training or deployment.

B Proofs

B.1 Piecewise-affine encoders with sparse supports induce local affine Koopman predictors

Once the encoder is piecewise affine, exact sparse supports induce a finite support-stratified collection of local affine decoded Koopman predictors. Throughout this subsection, unless explicitly stated otherwise, "support" means the exact zero support $\{i : E_i(x) \neq 0\}$. We use the convention $\operatorname{sign}(0) = 0$, so coordinatewise sign vectors take values in $\{-1, 0, +1\}^{d_z}$.

Throughout, write $[d_z] := \{1, \dots, d_z\}$. For a vector $v \in \mathbb{R}^{d_z}$ and a set $S \subseteq [d_z]$, v_S denotes the subvector with indices in S. For a matrix M, $M_{:,S}$ denotes the submatrix consisting of the columns indexed by S.

Definition 1 (Polyhedral stratum)**.** A polyhedral stratum in $\mathbb{R}^n$ is a set that can be written as the solution set of finitely many affine equalities and strict affine inequalities:

$$\mathcal{C} = \left\{ x \in \mathbb{R}^n : a_i^\top x = \alpha_i \text{ for } i \in I_=, \quad b_j^\top x < \beta_j \text{ for } j \in I_<, \quad c_k^\top x > \gamma_k \text{ for } k \in I_> \right\}.$$

The stratum may have dimension smaller than n. Empty sets are allowed in intermediate constructions, but are omitted from stratifications.

Definition 2 (Finite polyhedral stratification)**.** A finite polyhedral stratification of $\mathbb{R}^n$ is a finite collection $\mathscr{C} = \{\mathcal{C}_r\}_{r=1}^R$ of nonempty polyhedral strata such that $\mathbb{R}^n = \bigsqcup_{r=1}^R \mathcal{C}_r$, where $\bigsqcup$ denotes disjoint union.

Definition 3 (Piecewise-affine map)**.** A map $E : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$ is piecewise affine on a finite polyhedral stratification $\mathscr{C} = \{\mathcal{C}_r\}_{r=1}^R$ if, for each r, there exist $A_r \in \mathbb{R}^{d_z \times d_x}$ and $a_r \in \mathbb{R}^{d_z}$ such that $E(x) = A_r x + a_r$ for all $x \in \mathcal{C}_r$.

We will use two elementary closure facts.

Lemma 1 (Affine preimages of polyhedral strata)**.** *Let $L : \mathbb{R}^n \to \mathbb{R}^m$ be affine, say $L(x) = Mx + b$. If $\mathcal{P} \subseteq \mathbb{R}^m$ is a polyhedral stratum, then $L^{-1}(\mathcal{P})$ is either empty or a polyhedral stratum in $\mathbb{R}^n$. Consequently, if $\{\mathcal{P}_j\}_{j=1}^N$ is a finite polyhedral stratification of $\mathbb{R}^m$, then the nonempty sets $L^{-1}(\mathcal{P}_j)$, $j = 1, \dots, N$, form a finite polyhedral stratification of $\mathbb{R}^n$.*

Proof. Write $\mathcal{P}$ as

$$\mathcal{P} = \{ y \in \mathbb{R}^m : \ell_i(y) = 0 \text{ for } i \in I_=, \quad g_j(y) < 0 \text{ for } j \in I_<, \quad h_k(y) > 0 \text{ for } k \in I_> \},$$

where ℓ_i, g_j, h_k are affine functions. Then

$$L^{-1}(\mathcal{P}) = \{x \in \mathbb{R}^n : \ell_i(Mx+b) = 0 \text{ for } i \in I_=, g_j(Mx+b) < 0 \text{ for } j \in I_<, h_k(Mx+b) > 0 \text{ for } k \in I_>\}.$$

Each composed function $\ell_i(Mx+b)$, $g_j(Mx+b)$, and $h_k(Mx+b)$ is affine in x. Hence $L^{-1}(\mathcal{P})$ is a polyhedral stratum, unless the system is inconsistent, in which case it is empty.

If $\{\mathcal{P}_j\}_{j=1}^N$ is a stratification of $\mathbb{R}^m$, then the preimages $L^{-1}(\mathcal{P}_j)$ are pairwise disjoint because the $\mathcal{P}_j$'s are pairwise disjoint, and they cover $\mathbb{R}^n$ because the $\mathcal{P}_j$'s cover $\mathbb{R}^m$. Removing empty preimages therefore gives a finite polyhedral stratification of $\mathbb{R}^n$. □

Lemma 2 (Finite sign refinement by affine functions)**.** *Let $\mathcal{C} \subseteq \mathbb{R}^n$ be a polyhedral stratum, and let $g_1, \dots, g_N : \mathbb{R}^n \to \mathbb{R}$ be affine functions. For each sign vector $\rho = (\rho_1, \dots, \rho_N) \in \{-1, 0, +1\}^N$, define*

$$\mathcal{C}_\rho = \{x \in \mathcal{C} : g_j(x) < 0 \text{ if } \rho_j = -1, \quad g_j(x) = 0 \text{ if } \rho_j = 0, \quad g_j(x) > 0 \text{ if } \rho_j = +1\}.$$

Then the nonempty sets $\mathcal{C}_\rho$ form a finite polyhedral stratification of $\mathcal{C}$. On each such stratum, the signs of $g_1, \dots, g_N$ are constant.

Proof. Each $\mathcal{C}_\rho$ is obtained from $\mathcal{C}$ by adding finitely many affine equalities and strict affine inequalities. Hence every nonempty $\mathcal{C}_\rho$ is a polyhedral stratum. For each $x \in \mathcal{C}$, every number $g_j(x)$ is exactly one of negative, zero, or positive. Thus x belongs to exactly one of the sets $\mathcal{C}_\rho$. Since there are only 3^N possible sign vectors, the resulting stratification is finite. □

Theorem 1 (Piecewise-affine sparse encoders induce local affine Koopman predictors)**.** *Let $E : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$ be a single-valued encoder. Assume that E is piecewise affine on a finite polyhedral stratification $\mathscr{C} = \{\mathcal{C}_r\}_{r=1}^R$ of $\mathbb{R}^{d_x}$. Thus, for every r, there exist $A_r \in \mathbb{R}^{d_z \times d_x}$ and $a_r \in \mathbb{R}^{d_z}$ such that $E(x) = A_r x + a_r$ for all $x \in \mathcal{C}_r$.*

Let the decoder be affine, $D_\phi(z) = D_{\mathrm{dec}} z + b_{\mathrm{dec}}$, where $D_{\mathrm{dec}} \in \mathbb{R}^{d_x \times d_z}$ and $b_{\mathrm{dec}} \in \mathbb{R}^{d_x}$, and let $K \in \mathbb{R}^{d_z \times d_z}$ be the latent transition matrix. For $h \geq 1$, define the decoded h-step open-loop Koopman predictor

$$F_h(x) = D_{\mathrm{dec}} K^h E(x) + b_{\mathrm{dec}}.$$

Then the following statements hold.

1. *The map F_h is piecewise affine on the same stratification $\mathscr{C}$. More precisely, on each $\mathcal{C}_r$, $F_h(x) = M_{r,h} x + m_{r,h}$, where $M_{r,h} = D_{\mathrm{dec}} K^h A_r$ and $m_{r,h} = D_{\mathrm{dec}} K^h a_r + b_{\mathrm{dec}}$.*
2. *There is a finite polyhedral refinement of $\mathscr{C}$ on which the signed latent support $\operatorname{sign}(E(x)) \in \{-1, 0, +1\}^{d_z}$ is constant on every stratum. In particular, the unsigned support $\operatorname{supp}(E(x)) = \{i \in [d_z] : E_i(x) \neq 0\}$ is also constant on every refined stratum.*
3. *Let $\mathcal{S}$ be one of the refined strata from part 2, and let $S_{\mathcal{S}} = \operatorname{supp}(E(x))$ be the support on that stratum. Then, for all $x \in \mathcal{S}$,*

$$F_h(x) = D_{\mathrm{dec}} (K^h)_{:, S_{\mathcal{S}}} E_{S_{\mathcal{S}}}(x) + b_{\mathrm{dec}}.$$

Thus, on a fixed support stratum, only the active latent coordinates and the corresponding columns of K^h participate in the decoded predictor. For $h = 1$, these are precisely the corresponding columns of K.

4. *Fix a re-encoding period $p \geq 1$, and define the periodically re-encoded segment map $G_p(x) = D_{\mathrm{dec}} K^p E(x) + b_{\mathrm{dec}}$. Then G_p is piecewise affine on a finite polyhedral stratification. Moreover, for every finite number $T \geq 1$ of re-encoding segments, the T-fold composition*

$$G_p^{\circ T} = \underbrace{G_p \circ G_p \circ \cdots \circ G_p}_{T \text{ times}}$$

is piecewise affine on a finite polyhedral itinerary stratification. Each itinerary stratum is determined by the finite sequence of encoder regions visited by the rollout.

This theorem is an architectural statement. It shows that piecewise-affine encoders induce a finite collection of support-stratified affine decoded predictors. It does not by itself imply that supports recover basins, that the local affine predictors are accurate, or that the learned finite-dimensional coordinates are exactly Koopman-invariant.

Scope of the support identity. The identity in part 3 is an exact zero-support identity. It should not be read as an exact statement about thresholded or top-k support rules unless the omitted coordinates vanish. For any externally chosen set $S \subseteq [d_z]$, the exact decomposition is

$$F_h(x) = D_{\mathrm{dec}} (K^h)_{:,S} E_S(x) + D_{\mathrm{dec}} (K^h)_{:,S^c} E_{S^c}(x) + b_{\mathrm{dec}}.$$

Here $S^c = [d_z] \setminus S$. The omitted-coordinate contribution is zero whenever $E_{S^c}(x) = 0$, but the converse need not hold. In general,

$$D_{\mathrm{dec}} (K^h)_{:,S^c} E_{S^c}(x) = 0 \quad \Longleftrightarrow \quad E_{S^c}(x) \in \ker\Big(D_{\mathrm{dec}} (K^h)_{:,S^c} \Big).$$

Thus this vanishing is equivalent to $E_{S^c}(x) = 0$ only under the additional injectivity condition $\ker(D_{\mathrm{dec}} (K^h)_{:,S^c}) = \{0\}$. Moreover, the support selects the active columns of K^h, but the full affine state-space predictor also depends on the encoder stratum through $E_S(x) = (A_r x + a_r)_S$. Thus two refined strata can share the same support while inducing different affine maps in state space.

Proof. We prove the four claims in order.

Part 1. Fix a stratum $\mathcal{C}_r$. By assumption, $E(x) = A_r x + a_r$ for all $x \in \mathcal{C}_r$. Therefore, for $x \in \mathcal{C}_r$,

$$\begin{aligned} F_h(x) &= D_{\mathrm{dec}} K^h E(x) + b_{\mathrm{dec}} \\ &= D_{\mathrm{dec}} K^h (A_r x + a_r) + b_{\mathrm{dec}} \\ &= D_{\mathrm{dec}} K^h A_r x + D_{\mathrm{dec}} K^h a_r + b_{\mathrm{dec}}. \end{aligned}$$

Thus F_h is affine on $\mathcal{C}_r$, with $M_{r,h} = D_{\mathrm{dec}} K^h A_r$ and $m_{r,h} = D_{\mathrm{dec}} K^h a_r + b_{\mathrm{dec}}$. Since there are finitely many strata $\mathcal{C}_r$, the map F_h is piecewise affine on $\mathscr{C}$.

Part 2. Fix a stratum $\mathcal{C}_r$. On $\mathcal{C}_r$, the i-th coordinate of the encoder is the affine scalar function $E_i(x) = e_i^\top (A_r x + a_r)$, where e_i is the i-th standard basis vector of $\mathbb{R}^{d_z}$. Apply the finite sign-refinement lemma to the affine functions $g_{r,i}(x) = e_i^\top (A_r x + a_r)$, $i = 1, \ldots, d_z$. For each sign vector $\rho \in \{-1, 0, +1\}^{d_z}$, define

$$\mathcal{C}_{r,\rho} = \{ x \in \mathcal{C}_r : g_{r,i}(x) < 0 \text{ if } \rho_i = -1, \quad g_{r,i}(x) = 0 \text{ if } \rho_i = 0, \quad g_{r,i}(x) > 0 \text{ if } \rho_i = +1 \}.$$

The nonempty sets $\mathcal{C}_{r,\rho}$, over all r and ρ, form a finite polyhedral refinement of $\mathscr{C}$. Indeed, for each fixed r they stratify $\mathcal{C}_r$, and the original $\mathcal{C}_r$'s already stratify $\mathbb{R}^{d_x}$.

By construction, $\mathrm{sign}(E(x)) = \rho$ for all $x \in \mathcal{C}_{r,\rho}$. Hence the signed support is constant on every refined stratum. The unsigned support is obtained by forgetting the signs, $\mathrm{supp}(E(x)) = \{i : \rho_i \neq 0\}$, and is therefore also constant on every refined stratum.

Part 3. Let $\mathcal{S} = \mathcal{C}_{r,\rho}$ be one of the refined strata from part 2, and define $S_{\mathcal{S}} = \{i \in [d_z] : \rho_i \neq 0\}$. For every $x \in \mathcal{S}$, we have $E_i(x) = 0$ for all $i \notin S_{\mathcal{S}}$. Thus $E(x)$ has zero coordinates outside $S_{\mathcal{S}}$. Therefore $K^h E(x) = (K^h)_{:,S_{\mathcal{S}}} E_{S_{\mathcal{S}}}(x)$. Substituting this identity into the decoded predictor gives

$$F_h(x) = D_{\mathrm{dec}} K^h E(x) + b_{\mathrm{dec}} = D_{\mathrm{dec}} (K^h)_{:,S_{\mathcal{S}}} E_{S_{\mathcal{S}}}(x) + b_{\mathrm{dec}}.$$

This proves that the local predictor on $\mathcal{S}$ uses only the active latent coordinates and the corresponding columns of K^h.

Part 4. The map G_p is exactly F_p, so by part 1 it is piecewise affine on a finite polyhedral stratification. Let $\mathscr{P} = \{\mathcal{P}_1, \ldots, \mathcal{P}_N\}$ be such a stratification, and write $G_p(x) = B_j x + c_j$ for all $x \in \mathcal{P}_j$.

We prove that every finite composition $G_p^{\circ T}$ is piecewise affine by constructing itinerary strata. Define $G_p^{\circ 0}$ to be the identity map. For an itinerary $\alpha = (\alpha_0, \alpha_1, \ldots, \alpha_{T-1}) \in \{1, \ldots, N\}^T$, define the final itinerary cell

$$\Omega_\alpha = \left\{ x \in \mathbb{R}^{d_x} : G_p^{\circ s}(x) \in \mathcal{P}_{\alpha_s} \text{ for } s = 0, \ldots, T-1 \right\}.$$

The nonempty sets Ω_α cover $\mathbb{R}^{d_x}$, because every point has a unique stratum membership at each of the finitely many times $0, \ldots, T-1$. They are pairwise disjoint because the strata $\mathcal{P}_j$ are pairwise disjoint.

It remains to show that each nonempty Ω_α is polyhedral and that $G_p^{\circ T}$ is affine on it. For $t = 0, \ldots, T$, define the partial itinerary cell

$$\Omega_{\alpha,t} = \left\{ x \in \mathbb{R}^{d_x} : G_p^{\circ s}(x) \in \mathcal{P}_{\alpha_s} \text{ for } s = 0, \ldots, t-1 \right\},$$

with the convention that $\Omega_{\alpha,0} = \mathbb{R}^{d_x}$. Then $\Omega_{\alpha,T} = \Omega_\alpha$. We prove by induction on t that each nonempty $\Omega_{\alpha,t}$ is a polyhedral stratum and that $G_p^{\circ t}$ is affine on $\Omega_{\alpha,t}$.

For $t = 0$, $\Omega_{\alpha,0} = \mathbb{R}^{d_x}$, which is a polyhedral stratum, and $G_p^{\circ 0}(x) = x$ is affine.

Assume the claim holds for some $t < T$. Thus, on $\Omega_{\alpha,t}$, $G_p^{\circ t}(x) = L_t x + \ell_t$ for some matrix L_t and vector ℓ_t. The next partial cell is

$$\Omega_{\alpha,t+1} = \Omega_{\alpha,t} \cap \{x : L_t x + \ell_t \in \mathcal{P}_{\alpha_t}\}.$$

By the affine-preimage lemma, the second set is either empty or a polyhedral stratum. Its intersection with the polyhedral stratum $\Omega_{\alpha,t}$ is therefore either empty or a polyhedral stratum.

On any nonempty $\Omega_{\alpha,t+1}$, we have $G_p^{\circ t}(x) \in \mathcal{P}_{\alpha_t}$, and on $\mathcal{P}_{\alpha_t}$ the map G_p has the affine formula $G_p(y) = B_{\alpha_t} y + c_{\alpha_t}$. Therefore, for $x \in \Omega_{\alpha,t+1}$,

$$\begin{aligned} G_p^{\circ(t+1)}(x) &= G_p(G_p^{\circ t}(x)) \\ &= B_{\alpha_t}(L_t x + \ell_t) + c_{\alpha_t} \\ &= (B_{\alpha_t} L_t) x + (B_{\alpha_t} \ell_t + c_{\alpha_t}), \end{aligned}$$

which is affine in x. This completes the induction.

Taking $t = T$, each nonempty final cell Ω_α is polyhedral and $G_p^{\circ T}$ is affine on it. Since there are only finitely many itineraries $\alpha \in \{1, \ldots, N\}^T$, the nonempty final cells give a finite polyhedral stratification, and $G_p^{\circ T}$ is piecewise affine. □

Corollary 1 (Exact sparse coding under uniqueness)**.** *Let $B \in \mathbb{R}^{d_x \times d_z}$ and $\lambda > 0$, and, for each $x \in \mathbb{R}^{d_x}$, consider the lasso sparse-coding problem*

$$E_{\text{lasso}}(x) = z^\star(x) = \arg\min_{z \in \mathbb{R}^{d_z}} \left\{ \frac{1}{2}\|x - Bz\|_2^2 + \lambda \|z\|_1 \right\}.$$

Assume that the minimizer is unique for every $x \in \mathbb{R}^{d_x}$. Then E_{lasso} is single-valued, continuous, and piecewise affine on a finite polyhedral stratification. Consequently, the general theorem applies with $E = E_{\text{lasso}}$. In particular, for every $h \geq 1$, $x \mapsto D_{\text{dec}} K^h E_{\text{lasso}}(x) + b_{\text{dec}}$ is piecewise affine, and on every refined support stratum $\mathcal{S}$,

$$D_{\text{dec}} K^h E_{\text{lasso}}(x) + b_{\text{dec}} = D_{\text{dec}} (K^h)_{:,S_\mathcal{S}} (E_{\text{lasso}}(x))_{S_\mathcal{S}} + b_{\text{dec}}.$$

The uniqueness assumption is essential. It is satisfied, for example, if B has full column rank.

Proof. For fixed x, define $J_x(z) = \frac{1}{2}\|x - Bz\|_2^2 + \lambda\|z\|_1$. Because $\lambda > 0$, $J_x(z) \geq \lambda\|z\|_1$, and hence $J_x(z) \to \infty$ as $\|z\|_1 \to \infty$. Since J_x is continuous, a minimizer exists. By assumption, this minimizer is unique.

We first derive the optimality conditions from first principles. Let $r = x - Bz$. We claim that z minimizes J_x if and only if $B^\top(x - Bz) = \lambda\gamma$ for some $\gamma \in \mathbb{R}^{d_z}$ satisfying

$$\gamma_i = \begin{cases} \operatorname{sign}(z_i), & z_i \neq 0, \\ \text{some number in } [-1, 1], & z_i = 0. \end{cases}$$

First suppose z is a minimizer. Let b_i denote the i-th column of B. The differentiable part of J_x has coordinate derivative

$$\frac{\partial}{\partial z_i}\left(\frac{1}{2}\|x - Bz\|_2^2\right) = b_i^\top(Bz - x) = -b_i^\top r.$$

If $z_i > 0$, then for sufficiently small perturbations in the i-th coordinate the absolute-value term is differentiable with derivative $+1$. Optimality of z therefore gives $-b_i^\top r + \lambda = 0$, or equivalently $b_i^\top r = \lambda$. If $z_i < 0$, the absolute-value term has derivative -1, and optimality gives $-b_i^\top r - \lambda = 0$, or equivalently $b_i^\top r = -\lambda$. If $z_i = 0$, then moving in the positive coordinate direction by $t > 0$ gives

$$J_x(z + te_i) - J_x(z) = t(-b_i^\top r + \lambda) + O(t^2).$$

Since z is a minimizer, the first-order coefficient must be nonnegative: $-b_i^\top r + \lambda \geq 0$, so $b_i^\top r \leq \lambda$. Moving in the negative coordinate direction gives

$$J_x(z - te_i) - J_x(z) = t(b_i^\top r + \lambda) + O(t^2),$$

and hence $b_i^\top r + \lambda \geq 0$, so $b_i^\top r \geq -\lambda$. Thus, when $z_i = 0$, $|b_i^\top r| \leq \lambda$. Combining the three cases gives the claimed condition.

Conversely, suppose the claimed condition holds. Let $w \in \mathbb{R}^{d_z}$ be arbitrary. Since $r = x - Bz$, we have $x - Bw = x - Bz - B(w - z) = r - B(w - z)$. Therefore

$$\begin{aligned} J_x(w) - J_x(z) &= \frac{1}{2}\|r - B(w - z)\|_2^2 - \frac{1}{2}\|r\|_2^2 + \lambda(\|w\|_1 - \|z\|_1) \\ &= -r^\top B(w - z) + \frac{1}{2}\|B(w - z)\|_2^2 + \lambda(\|w\|_1 - \|z\|_1). \end{aligned}$$

The condition $B^\top r = \lambda\gamma$ gives $r^\top B(w-z) = (B^\top r)^\top (w-z) = \lambda\gamma^\top(w-z)$. Hence

$$J_x(w) - J_x(z) = \frac{1}{2}\|B(w-z)\|_2^2 + \lambda\big(\|w\|_1 - \|z\|_1 - \gamma^\top(w-z)\big).$$

For each coordinate i, the definition of γ_i gives $|w_i| - |z_i| \geq \gamma_i(w_i - z_i)$. Indeed, if $z_i > 0$, then $\gamma_i = 1$ and this inequality is $|w_i| - z_i \geq w_i - z_i$, which follows from $|w_i| \geq w_i$. If $z_i < 0$, then $\gamma_i = -1$ and the inequality is $|w_i| + z_i \geq -w_i + z_i$, which follows from $|w_i| \geq -w_i$. If $z_i = 0$, then $|\gamma_i| \leq 1$, so $|w_i| \geq \gamma_i w_i$. Summing over coordinates gives $\|w\|_1 - \|z\|_1 \geq \gamma^\top(w-z)$. Thus $J_x(w) - J_x(z) \geq \frac{1}{2}\|B(w-z)\|_2^2 \geq 0$. Since w was arbitrary, z minimizes J_x. This proves the optimality criterion.

We now construct the affine pieces. Let $A \subseteq [d_z]$ be a candidate active set and let $s \in \{-1,+1\}^A$ be a candidate sign vector. We use the convention that, when $A = \varnothing$, all matrices and vectors restricted to A have the corresponding empty dimensions and the formula below gives the zero vector.

Assume $B_A^\top B_A$ is invertible. Define the affine candidate

$$z_A^{A,s}(x) = (B_A^\top B_A)^{-1}(B_A^\top x - \lambda s), \qquad z_{A^c}^{A,s}(x) = 0.$$

Define the corresponding residual $r_{A,s}(x) = x - B_A z_A^{A,s}(x)$. Now define

$$P_{A,s} = \left\{ x \in \mathbb{R}^{d_x} : s_i z_i^{A,s}(x) \geq 0 \text{ for all } i \in A, \quad |b_j^\top r_{A,s}(x)| \leq \lambda \text{ for all } j \notin A \right\}.$$

The functions $z_i^{A,s}(x)$ and $b_j^\top r_{A,s}(x)$ are affine in x. Hence $P_{A,s}$ is a closed polyhedron.

For $x \in P_{A,s}$, the active residual equation holds by construction:

$$B_A^\top r_{A,s}(x) = B_A^\top x - B_A^\top B_A z_A^{A,s}(x) = \lambda s.$$

For inactive indices $j \notin A$, the definition of $P_{A,s}$ gives $|b_j^\top r_{A,s}(x)| \leq \lambda$. For active indices $i \in A$, if $z_i^{A,s}(x) \neq 0$, then $s_i z_i^{A,s}(x) > 0$, so $\operatorname{sign}(z_i^{A,s}(x)) = s_i$. If $z_i^{A,s}(x) = 0$, then $s_i \in [-1,1]$, which is allowed by the zero-coordinate optimality condition. Therefore the optimality criterion above shows that $z^{A,s}(x)$ is a minimizer of J_x. Since the minimizer is unique, $E_{\text{lasso}}(x) = z^{A,s}(x)$ for all $x \in P_{A,s}$. Thus E_{lasso} is affine on each such polyhedron.

It remains to show that these finitely many polyhedra cover $\mathbb{R}^{d_x}$. Fix x, and let $z^\star = E_{\text{lasso}}(x)$, $A = \operatorname{supp}(z^\star)$, and $s = \operatorname{sign}(z_A^\star)$. We claim that $B_A^\top B_A$ must be invertible. If $A = \varnothing$, this is true by convention. Otherwise, suppose $B_A^\top B_A$ is singular. Then there exists a nonzero vector $h \in \mathbb{R}^{|A|}$ such that $B_A h = 0$. Let $r^\star = x - B_A z_A^\star$. The active optimality equations give $B_A^\top r^\star = \lambda s$. Multiplying by h, we obtain

$$\lambda s^\top h = h^\top B_A^\top r^\star = (B_A h)^\top r^\star = 0.$$

Thus $s^\top h = 0$. For sufficiently small nonzero t, the vector $z_A^\star + th$ has the same coordinate signs as $z_A^\star$. Also,

$$B_A(z_A^\star + th) = B_A z_A^\star + tB_A h = B_A z_A^\star,$$

so the residual is unchanged. Since the signs are unchanged,

$$\|z_A^\star + th\|_1 = s^\top(z_A^\star + th) = s^\top z_A^\star + ts^\top h = s^\top z_A^\star = \|z_A^\star\|_1.$$

Therefore $z_A^\star + th$, extended by zeros on A^c, gives the same objective value as $z^\star$. This contradicts uniqueness. Hence $B_A^\top B_A$ is invertible.

Since $B_A^\top B_A$ is invertible, the active optimality equation $B_A^\top(x - B_A z_A^\star) = \lambda s$ implies

$$z_A^\star = (B_A^\top B_A)^{-1}(B_A^\top x - \lambda s) = z_A^{A,s}(x).$$

The inactive optimality inequalities give $|b_j^\top r_{A,s}(x)| \le \lambda$ for all $j \notin A$, and the sign conditions give $s_i z_i^{A,s}(x) > 0$ for all $i \in A$. Thus $x \in P_{A,s}$. Therefore the finite collection of polyhedra $P_{A,s}$, over all $A \subseteq [d_z]$ and $s \in \{-1,+1\}^A$ with $B_A^\top B_A$ invertible, covers $\mathbb{R}^{d_x}$.

The sets $P_{A,s}$ may overlap on boundaries. To obtain a genuine stratification, take the finite family of all affine functions appearing in the defining inequalities of all the polyhedra $P_{A,s}$, and stratify $\mathbb{R}^{d_x}$ according to the signs of those functions. By the finite sign-refinement lemma, this gives a finite polyhedral stratification. On every stratum, membership in each $P_{A,s}$ is constant. Since the $P_{A,s}$'s cover $\mathbb{R}^{d_x}$, each stratum lies inside at least one $P_{A,s}$. On that stratum, $E_{\text{lasso}}(x) = z^{A,s}(x)$, which is affine in x. If a stratum lies inside more than one such polyhedron, the corresponding affine formulas agree on the stratum because they all equal the unique minimizer. Hence E_{lasso} is piecewise affine on a finite polyhedral stratification.

The general theorem now applies directly with $E = E_{\text{lasso}}$. Therefore the decoded h-step Koopman predictor $x \mapsto D_{\text{dec}} K^h E_{\text{lasso}}(x) + b_{\text{dec}}$ is piecewise affine, and on each refined support stratum $\mathcal{S}$ it uses only the active coordinates:

$$D_{\text{dec}} K^h E_{\text{lasso}}(x) + b_{\text{dec}} = D_{\text{dec}} (K^h)_{:,S_\mathcal{S}} (E_{\text{lasso}}(x))_{S_\mathcal{S}} + b_{\text{dec}}.$$

We finally prove continuity of E_{lasso}. Let $x_n \to x$, and set $z_n = E_{\text{lasso}}(x_n)$. Since z_n minimizes J_{x_n},

$$\frac{1}{2}\|x_n - Bz_n\|_2^2 + \lambda\|z_n\|_1 \le \frac{1}{2}\|x_n\|_2^2.$$

In particular, $\lambda\|z_n\|_1 \le \frac{1}{2}\|x_n\|_2^2$. The sequence (x_n) is bounded, so (z_n) is bounded. Let (z_{n_k}) be any convergent subsequence, and write $z_{n_k} \to \bar{z}$. For every $w \in \mathbb{R}^{d_z}$, optimality of z_{n_k} gives

$$J_{x_{n_k}}(z_{n_k}) \le J_{x_{n_k}}(w).$$

Taking $k \to \infty$, using continuity of $J_x(z)$ jointly in (x, z), gives

$$J_x(\bar{z}) \le J_x(w) \qquad \text{for all } w.$$

Thus $\bar{z}$ is a minimizer of J_x. By uniqueness, $\bar{z} = E_{\text{lasso}}(x)$. Every convergent subsequence of (z_n) has the same limit $E_{\text{lasso}}(x)$, and (z_n) is bounded. Therefore $z_n \to E_{\text{lasso}}(x)$. So E_{lasso} is continuous.

If B has full column rank, then $B^\top B$ is positive definite, so the quadratic term $z \mapsto \frac{1}{2}\|x - Bz\|_2^2$ is strictly convex. The term $\lambda\|z\|_1$ is convex. The sum of a strictly convex function and a convex function is strictly convex, so J_x has at most one minimizer. Since a minimizer exists, it is unique. Hence full column rank is a sufficient condition for the uniqueness assumption. □

Corollary 2 (Finite-depth LISTA encoders)**.** *Let $Q \ge 0$ be an integer. Let $c : \mathbb{R}^{d_x} \to \mathbb{R}^{d_z}$ be a single-valued pre-code that is piecewise affine on a finite polyhedral stratification. This includes the affine special case $c(x) = Wx + b$, where $W \in \mathbb{R}^{d_z \times d_x}$ and $b \in \mathbb{R}^{d_z}$. Let $S_{\text{L}} \in \mathbb{R}^{d_z \times d_z}$, let $\tau \in \mathbb{R}^{d_z}_{\ge 0}$, and define the coordinatewise soft-thresholding map by $[T_\tau(y)]_i = \operatorname{sign}(y_i)\max\{|y_i| - \tau_i, 0\}$. Consider the finite-depth LISTA encoder*

$$u^{(0)}(x) = T_\tau(c(x)),$$
$$u^{(q+1)}(x) = T_\tau(S_{\text{L}} u^{(q)}(x) + c(x)), \qquad q = 0, \dots, Q-1,$$

and

$$E_{\mathrm{LISTA}}(x) = u^{(Q)}(x).$$

Here Q counts the number of refinement steps after the initial thresholding; when $Q = 0$, the encoder is simply $E_{\mathrm{LISTA}} = u^{(0)}$. Then E_{LISTA} is single-valued and piecewise affine on a finite polyhedral stratification of $\mathbb{R}^{d_x}$. Consequently, the general theorem applies with $E = E_{\mathrm{LISTA}}$. In particular, for every $h \geq 1$, $x \mapsto D_{\mathrm{dec}} K^h E_{\mathrm{LISTA}}(x) + b_{\mathrm{dec}}$ is piecewise affine, and on every refined support stratum $\mathcal{S}$,

$$D_{\mathrm{dec}} K^h E_{\mathrm{LISTA}}(x) + b_{\mathrm{dec}} = D_{\mathrm{dec}} (K^h)_{:,S_{\mathcal{S}}} (E_{\mathrm{LISTA}}(x))_{S_{\mathcal{S}}} + b_{\mathrm{dec}}.$$

Proof. We first show that the soft-thresholding map T_τ is piecewise affine on a finite polyhedral stratification of $\mathbb{R}^{d_z}$.

For one coordinate, define $t_i(y_i) = \mathrm{sign}(y_i) \max\{|y_i| - \tau_i, 0\}$. Equivalently,

$$t_i(y_i) = \begin{cases} y_i + \tau_i, & y_i < -\tau_i, \\ 0, & -\tau_i \leq y_i \leq \tau_i, \\ y_i - \tau_i, & y_i > \tau_i. \end{cases}$$

To obtain a disjoint stratification, split the real line into the nonempty and distinct sets among $(-\infty, -\tau_i)$, $\{-\tau_i\}$, $(-\tau_i, \tau_i)$, $\{\tau_i\}$, and (τ_i, ∞). The phrase "distinct" only matters when $\tau_i = 0$, in which case $\{-\tau_i\} = \{\tau_i\}$ and $(-\tau_i, \tau_i)$ is empty. On each remaining piece, t_i is affine: $t_i(y_i) = y_i + \tau_i$ on $(-\infty, -\tau_i)$, $t_i(y_i) = 0$ on the pieces contained in $[-\tau_i, \tau_i]$, and $t_i(y_i) = y_i - \tau_i$ on (τ_i, ∞). Taking Cartesian products of these one-dimensional strata over $i = 1, \ldots, d_z$ gives a finite polyhedral stratification of $\mathbb{R}^{d_z}$. On each product stratum $\mathcal{P}$, there exist a diagonal matrix $R_{\mathcal{P}}$ and a vector $r_{\mathcal{P}}$ such that $T_\tau(y) = R_{\mathcal{P}} y + r_{\mathcal{P}}$ for all $y \in \mathcal{P}$. Thus T_τ is piecewise affine.

Let $\mathscr{C}^c = \{\mathcal{C}^c_\alpha\}_\alpha$ be a finite polyhedral stratification on which the pre-code c is affine. Thus, on each $\mathcal{C}^c_\alpha$, there exist W^c_α and b^c_α such that $c(x) = W^c_\alpha x + b^c_\alpha$ for all $x \in \mathcal{C}^c_\alpha$. If $c(x) = Wx + b$ is globally affine, this stratification has one stratum.

We now prove by induction over the LISTA depth that every $u^{(q)}$ is piecewise affine on a finite polyhedral stratification that refines $\mathscr{C}^c$.

For the initial step, let $\mathcal{P}$ be one of the threshold strata for T_τ. On a fixed pre-code stratum $\mathcal{C}^c_\alpha$, the set

$$\mathcal{C}^c_\alpha \cap \{x : W^c_\alpha x + b^c_\alpha \in \mathcal{P}\}$$

is either empty or a polyhedral stratum: it is the intersection of $\mathcal{C}^c_\alpha$ with the affine preimage of $\mathcal{P}$. The nonempty sets of this form, over all α and all threshold strata $\mathcal{P}$, form a finite polyhedral stratification of $\mathbb{R}^{d_x}$ refining $\mathscr{C}^c$. On such a stratum,

$$\begin{aligned} u^{(0)}(x) &= T_\tau(c(x)) \\ &= R_{\mathcal{P}}(W^c_\alpha x + b^c_\alpha) + r_{\mathcal{P}} \\ &= (R_{\mathcal{P}} W^c_\alpha) x + (R_{\mathcal{P}} b^c_\alpha + r_{\mathcal{P}}), \end{aligned}$$

which is affine in x. Therefore $u^{(0)}$ is piecewise affine.

Now assume that, for some $q \geq 0$, $u^{(q)}$ is piecewise affine on a finite polyhedral stratification $\mathscr{C}^{(q)} = \{\mathcal{C}^{(q)}_\beta\}_\beta$ that refines $\mathscr{C}^c$. Thus, on each stratum $\mathcal{C}^{(q)}_\beta$, there exist $A^{(q)}_\beta$ and $a^{(q)}_\beta$ such that $u^{(q)}(x) = A^{(q)}_\beta x + a^{(q)}_\beta$. Because $\mathscr{C}^{(q)}$ refines $\mathscr{C}^c$, the pre-code is also affine on $\mathcal{C}^{(q)}_\beta$; write $c(x) =$

$W_\beta^c x + b_\beta^c$ for all $x \in \mathcal{C}_\beta^{(q)}$. On this same stratum, the next pre-threshold vector is

$$\begin{aligned} v^{(q+1)}(x) &= S_{\mathrm{L}} u^{(q)}(x) + c(x) \\ &= S_{\mathrm{L}}(A_\beta^{(q)} x + a_\beta^{(q)}) + W_\beta^c x + b_\beta^c \\ &= (S_{\mathrm{L}} A_\beta^{(q)} + W_\beta^c) x + (S_{\mathrm{L}} a_\beta^{(q)} + b_\beta^c), \end{aligned}$$

which is affine in x.

Let $\mathcal{P}$ again be a threshold stratum for T_τ. The set

$$\mathcal{C}_\beta^{(q)} \cap \{x : v^{(q+1)}(x) \in \mathcal{P}\}$$

is either empty or a polyhedral stratum: it is the intersection of $\mathcal{C}_\beta^{(q)}$ with the affine preimage of $\mathcal{P}$. These nonempty intersections, over all β and all threshold strata $\mathcal{P}$, give a finite polyhedral refinement of $\mathscr{C}^{(q)}$, and hence still refine $\mathscr{C}^c$. On such a refined stratum,

$$\begin{aligned} u^{(q+1)}(x) &= T_\tau(v^{(q+1)}(x)) \\ &= R_{\mathcal{P}} v^{(q+1)}(x) + r_{\mathcal{P}} \\ &= R_{\mathcal{P}}\big((S_{\mathrm{L}} A_\beta^{(q)} + W_\beta^c) x + (S_{\mathrm{L}} a_\beta^{(q)} + b_\beta^c)\big) + r_{\mathcal{P}}, \end{aligned}$$

which is affine in x. Thus $u^{(q+1)}$ is piecewise affine on a finite polyhedral stratification that refines $\mathscr{C}^c$.

By induction, $E_{\mathrm{LISTA}}(x) = u^{(Q)}(x)$ is piecewise affine on a finite polyhedral stratification. The encoder is single-valued because it is defined by deterministic compositions of functions.

The general theorem now applies with $E = E_{\mathrm{LISTA}}$. Hence $x \mapsto D_{\mathrm{dec}} K^h E_{\mathrm{LISTA}}(x) + b_{\mathrm{dec}}$ is piecewise affine, and on each refined support stratum $\mathcal{S}$,

$$D_{\mathrm{dec}} K^h E_{\mathrm{LISTA}}(x) + b_{\mathrm{dec}} = D_{\mathrm{dec}} (K^h)_{:,S_{\mathcal{S}}} (E_{\mathrm{LISTA}}(x))_{S_{\mathcal{S}}} + b_{\mathrm{dec}}.$$

When the pre-code is written as $c(x) = f_\theta(x)$, this corollary applies literally whenever f_θ is piecewise affine, for example an affine, ReLU, or leaky-ReLU MLP. It does not automatically cover smooth pre-codes such as tanh, sigmoid, GELU, or SiLU networks, or input-dependent normalizations, unless a separate approximation or regularity argument is supplied. □

C Additional experimental details

Training and model rows. The fixed multibasin comparison uses the five model rows reported in table 1. All are trained on length-8 observation windows for 200,000 optimization steps with minibatches of 256 windows, using the prediction, reconstruction, latent-consistency, and sparsity objective described in section 4.5. The multibasin runs use AdamW, with learning rate 5×10^{-5} for the encoder and decoder, learning rate 5×10^{-6} for the latent transition, and weight decay 10^{-4} on the non-transition parameters. In the notation of equation (15), the relative weights are $\lambda_{\mathrm{pred}} = 1$, $\lambda_{\mathrm{rec}} = 0.03$, and $\lambda_{\mathrm{lin}} = 1$; sparse rows use $\lambda_{\mathrm{sp}} = 3\times10^{-3}$, while the Dense MLP row sets $\lambda_{\mathrm{sp}} = 0$. The boundary-emphasized training distribution draws half of its resets from a pool of hard initial conditions selected without basin labels, using short probe rollouts to identify states with high perturbation sensitivity or lingering transient motion.

The saved best checkpoint is chosen by a fixed validation rule during training. Every 500 optimization steps, and again at the final step, the current model is rolled out for 200 steps from 16

held-out initial states using every-step re-encoding. The checkpoint with the lowest final validation error is saved as `checkpoint.pt`; the latest checkpoint is kept separately as `last.pt`. This validation rule is a training-side proxy for rollout stability. It does not use basin labels, test trajectories, or the post-hoc best-periodic evaluation grid.

The main Dysts comparison is used only as a forecasting stress test and includes LISTA, LISTA-BD, LISTA-SB, Sparse MLP, Sparse MLP-BD, and Dense MLP rows. Dysts training trajectories are drawn from native trajectory caches with standardized coordinates, 200 cached trajectories, 30,000 stored observations per trajectory, and a 2,000-step warm-up; training samples length-10 windows from the cache training split. The current table reports the $dt\times30$ version of this benchmark: the stored Dysts observation interval is set to 30 times the intrinsic Dysts timestep for the corresponding system, and evaluation uses reduced horizon indices through $H5000$ rather than the older tiny-step $H\leq60000$ extrapolation packet. Initial conditions are based on each Dysts system's default initial condition; the remaining cached trajectories perturb that state with Gaussian noise at scale 0.2 times the coordinate-wise Dysts standard deviation. Train, validation, and test caches use deterministic split-specific namespaces so that held-out trajectories are disjoint from training windows and reusable across model seeds. Long-horizon evaluation uses the disjoint held-out test cache, with 100 held-out trajectories for each system and seed. LISTA rows use the same learning rates as the multibasin runs, while MLP rows use learning rates 10^{-4} for encoder/decoder parameters and 10^{-5} for the latent transition. The paper-facing Dysts shortlist contains the 12 systems in table 6.

Table 6: Dysts forecasting shortlist. These systems are used only for long-horizon forecasting stress tests because basin labels are not available.

System	State dimension
`Chua`	3
`Dadras`	3
`DequanLi`	3
`Hadley`	3
`LorenzCoupled`	6
`LuChenCheng`	3
`MultiChua`	3
`QiChen`	3
`Sakarya`	3
`SanUmSrisuchinwong`	3
`ShimizuMorioka`	3
`WangSun`	3

The Dysts systems have system-specific native integration intervals, so the cached sequences should be interpreted as benchmark observation sequences rather than as trajectories normalized to a common physical time. Coordinates are standardized before training and evaluation. The models never observe the vector field, the continuous solver, or the native integration substeps; they receive uniformly spaced stored states and are evaluated in stored-step units. This is why the Dysts table reports stored-step horizons such as $H1000$ and $H5000$ rather than a shared physical-time horizon.

Training recipe selection. Earlier exploratory sweeps varied LISTA depth, shrinkage scale, sparsity coefficient, sign-split encodings, transition structure, and MLP sparsity controls. Those pilots used shorter budgets or broader system lists, so their numeric grids are not treated as final

protocol here. The paper-facing appendix instead reports the fixed model rows, training budgets, validation rule, held-out splits, and statistical units used for the final claims. In particular, staged recipe-discovery sweeps with 10,000-step, three-seed cells and one-step training windows were used only to choose candidate recipes. They are not pooled with the final 200,000-step, held-out-split evaluations and should not be interpreted as paper evidence.

Evaluation splits. All reported forecasting and interpretability results are evaluated on held-out trajectories, not on the training windows. Multibasin forecasting uses 100 held-out initial conditions for each system and seed. Routing fits local predictors on 256 disjoint held-out trajectories of length 256 and evaluates them on a separate set of 128 held-out rollouts. The Dysts long-horizon table uses 100 held-out cached test trajectories for each system and seed. Basin labels, when available in the procedural benchmark, are used only to define evaluation slices, score support–basin agreement, construct controlled transfer pairs, and compute statistical summaries.

Periodic forecasting. All horizons and re-encoding periods are reported in stored observation steps, not in equal physical time across systems. The no-reencoding rollout applies the learned latent transition for the full horizon before decoding. The periodic rollout decodes and re-encodes the model's own predicted state at a fixed period. The multibasin forecasting tables select the best score over periods $10, 25, 50$, and 100 stored observation steps. The Dysts $dt\times30$ evaluation uses periods $10, 25, 50, 100, 150$, and 200. These choices are fixed before aggregation and do not use the true future trajectory.

The fixed re-encoding grid is used only at evaluation time. Checkpoint selection uses the validation proxy described above, not a search over the final test-period grid, and the reported point estimates aggregate all completed seeds rather than selecting the best seed. The best-periodic score should therefore be read as a controlled diagnostic of whether a model benefits from regular support refresh at deployment, not as teacher forcing or basin-label access.

Support diagnostics. The wrong-support ablation first constructs, for each basin in a testable system, a canonical deep-slice support from the most common exact support observed in that basin. The ablated rollout replaces the inferred support with a canonical support from a different basin and holds that replacement fixed. Table 1 reports the wrong-support/base error ratio at horizons $h = 1$ and $h = 20$; larger ratios indicate stronger functional dependence on the selected support.

Routing details. Support-routed forecasting uses fixed-size top-8 supports so every state receives a route key. The three routed predictors in table 2 are: a support-gated global transition, which masks the centered latent state by the current support before applying the learned transition; a centered support-local transition, which fits a separate ridge-regularized linear map for each fitted support; and a centered family-local transition, which uses the same local fitting procedure after merging nearby supports into support families. The ridge penalty is 10^{-4}. At evaluation time, a local route is used only if it was observed in at least 128 fitted transitions; otherwise the rollout falls back to the global latent transition. The reported routed/global ratio always compares the routed prediction with the same model's own global rollout, not with another model family.

Support refresh. The support-refresh experiment uses top-8 supports and the two re-encoding periods reported in table 3, 1 and 10 stored observation steps. Each comparison starts from a controlled source-to-target transfer. The frozen-source rollout keeps using the source support after target entry, while the refreshed-current rollout periodically decodes, re-encodes, and re-extracts the

Table 7: Partition-control local forecasting for the top-8 support-family comparison. Entries are routed/global MSE-ratio IQMs with within-system Wilcoxon/Holm counts in brackets; lower is better. The LISTA-SB row in this auxiliary control table uses the p64 diagnostic artifact, while the main routing table uses the matched d_z=256 LISTA-SB row.

		$H100$		$H1000$	
Model	Selector	All	Deep	All	Deep
LISTA-SB (p64)	Support family	0.212 [10/17]	0.0644 [13/17]	20.9 [13/17]	1.46 [14/17]
	Basin labels	0.127 [11/17]	0.117 [11/17]	7.7×10^{11} [11/17]	1.2×10^{14} [10/17]
	Latent clusters	1.3 [3/17]	0.308 [3/17]	1.51 [16/17]	0.00335 [16/17]
	Random matched	0.47 [16/17]	0.45 [14/17]	3.9×10^{-6} [17/17]	6.2×10^{-7} [16/17]
LISTA-BD	Support family	0.156 [12/17]	0.0378 [13/17]	0.0131 [15/17]	9.3×10^{-5} [15/17]
	Basin labels	0.00696 [16/17]	0.00347 [14/17]	0.371 [15/17]	1.67 [14/17]
	Latent clusters	0.206 [11/17]	0.0559 [13/17]	5.6×10^{-12} [16/17]	3.8×10^{-14} [16/17]
	Random matched	0.866 [15/17]	0.899 [14/17]	0.00839 [16/17]	0.00595 [15/17]
Sparse MLP, BD	Support family	0.222 [7/17]	0.101 [8/17]	200 [17/17]	0.228 [16/17]
	Basin labels	0.00391 [8/17]	0.00426 [7/17]	0.0359 [13/17]	0.0336 [11/17]
	Latent clusters	0.0139 [13/17]	0.00882 [14/17]	9.5×10^{-9} [17/17]	7.9×10^{-10} [16/17]
	Random matched	0.951 [12/17]	0.978 [10/17]	0.428 [8/17]	0.023 [6/17]
Sparse MLP	Support family	0.13 [6/17]	0.0605 [5/17]	0.375 [17/17]	0.28 [16/17]
	Basin labels	0.00369 [10/17]	0.00366 [10/17]	0.084 [13/17]	9.8×10^{-4} [12/17]
	Latent clusters	0.0133 [14/17]	0.0114 [13/17]	2×10^{-14} [17/17]	2.3×10^{-13} [16/17]
	Random matched	0.966 [14/17]	0.982 [13/17]	8.8×10^{10} [6/17]	0.189 [4/17]
Dense MLP	Support family	1×10^{3} [0/17]	662 [0/17]	675 [1/17]	533 [1/17]
	Basin labels	10.6 [5/17]	9.32 [4/17]	1.6×10^{16} [3/17]	1.8×10^{12} [2/17]
	Latent clusters	1.6×10^{6} [1/17]	1.3×10^{5} [3/17]	9.8×10^{11} [3/17]	7.6×10^{6} [3/17]
	Random matched	1.09 [1/17]	1 [1/17]	0.752 [2/17]	0.728 [1/17]

support from its own predicted state. Basin labels are used only to construct valid transfer pairs and score post-entry routing; they are not supplied to the rollout.

C.1 Statistical testing protocol

All bracketed K/N counts are within-system reproducibility counts, not tests obtained by pooling all seeds or all systems together. For each table cell and each benchmark system, we first construct paired deltas over the appropriate replicate unit. For multibasin forecasting, Dysts within-system reproducibility counts, S_{abs}-size diagnostics, support-family entropy, and wrong-support ablations, the replicate unit is the training seed and the pairing is by the same system and seed. For the aggregate Dysts model-vs-Dense test, the replicate unit is instead the benchmark system after seed aggregation: each system contributes one $\log_{10}$ ratio between the candidate seed-IQM and the Dense MLP seed-IQM. For non-oracle routing, the pairing is also by system and seed, but the comparator is the same trained model's own global latent rollout for the same evaluation subset and horizon. For support refresh, the replicate unit is the controlled source-to-target transfer instance within a system; when multiple training seeds are available, all completed seeds contribute transfer instances to the same within-system test.

The tested delta is chosen so that the sign matches the scientific claim. Forecasting comparisons

against the Dense MLP baseline use

$$\Delta_{s,u} = \log_{10} E^{\text{cand}}_{s,u} - \log_{10} E^{\text{base}}_{s,u} = \log_{10}\Big(E^{\text{cand}}_{s,u}/E^{\text{base}}_{s,u}\Big), \tag{28}$$

where s indexes the system and u indexes the replicate unit; the one-sided alternative is $\Delta < 0$. Routed/global and refreshed/previous comparisons use the same log-ratio convention, again with alternative $\Delta < 0$. Mean S_{abs} size and $H(B \mid F_{\text{abs}})$ use raw paired differences against the Dense MLP baseline with alternative candidate $<$ baseline. Exact-support uniqueness and wrong-support/base ratios use raw paired differences with alternative candidate $>$ baseline, because larger values mean that the representation assigns more unique basin-specific supports or depends more strongly on the selected support.

Within each system, we apply a one-sided paired Wilcoxon signed-rank test to these deltas. Systems with too few finite pairs, all-zero deltas, or diagnostics that are undefined by construction are treated as non-significant for that cell. For each model–metric–horizon–subset cell, the resulting per-system p-values are Holm-corrected across the systems in that family at $\alpha = 0.05$. The reported K/N is the number of systems whose Holm-corrected p-value is below 0.05 divided by the number of systems eligible for that diagnostic: 17 for the controlled multibasin benchmark, 12 for the Dysts shortlist, and 13 for the global deep-slice wrong-support ablation because four systems have a single-basin deep subset. The aggregate Dysts table uses a separate row-level test: the 12 per-system $\log_{10}$ candidate/Dense seed-IQM ratios are tested with a one-sided Wilcoxon signed-rank test and Holm-corrected across all candidate-model and horizon comparisons. Point estimates are kept separate from the tests: the tables report IQM summaries for robustness to heavy tails, the K/N values report per-system paired significance, and the aggregate Dysts ratio table reports model-vs-Dense inference at the same system-aggregated level as the Dysts point estimate.

Censored routing comparisons. Table 2 has an additional issue: at long horizons, either the routed rollout or the same-model global rollout can be invalid because no finite H-step error remains available for that seed and subset. The displayed routed/global ratio uses only finite positive routed/global ratios. The significance test, however, must still account for all 15 seed slots per system rather than silently dropping the hardest seeds. For each horizon, we choose a log-scale censoring cap

$$C = 1 + \left\lceil \max\left(12,\ \max_{\text{finite}} \left|\log_{10}(E^{\text{routed}}/E^{\text{global}})\right|\right)\right\rceil, \tag{29}$$

so the cap lies outside the observed finite log-ratio range. If both routed and global MSEs are finite and positive, the test uses the ordinary log ratio. If the routed MSE is finite but the same-model global MSE is invalid, the seed is encoded as a censored routed win, $\Delta = -C$. If the routed MSE is invalid but the global MSE is finite, the seed is encoded as a censored routed loss, $\Delta = +C$. If both sides are invalid or the expected seed row is missing, the seed is encoded as neutral, $\Delta = 0$. The same convention treats zero routed error against positive global error as a censored win, positive routed error against zero global error as a censored loss, and both-zero as neutral.

This censoring rule is an evaluation convention, not a causal diagnosis of why a rollout became invalid. It assumes only that, for an H-step forecasting benchmark, a finite evaluable forecast is better than an unevaluable forecast for the same model, system, seed, subset, and horizon. Finite catastrophic errors are retained as finite values; they are not discarded. Invalid global rollouts with finite routed rollouts therefore contribute directional evidence that routing produced an evaluable forecast where the same-model global rollout did not, but the statistic does not claim that the underlying cause was a particular learning failure. Encoding both-invalid and missing seed slots as neutral prevents the test from rewarding methods merely because both compared rollouts failed.

C.2 Compute resources

Reported multibasin and Dysts training jobs were launched as independent cluster array tasks requesting one GPU, four CPU cores, and 16GB memory per run, with a 24-hour wall-time cap. The routing and refresh diagnostics were run as CPU jobs requesting four CPU cores and 24GB memory, with a 12-hour wall-time cap. The Dysts long-horizon re-evaluation was run in 100-trajectory batches, with evaluation tasks requesting four CPU cores and 24GB memory. These values describe the requested resources for the reported packets; the wall-time caps are not measured runtimes.

D Support definitions and sensitivity

The absolute-threshold support S_{abs} marks a latent coordinate active when its magnitude exceeds 10^{-3}. It is the strict identifiability object used on the deep-state slice. Its active-coordinate count $|S_{\text{abs}}|$ is a state-dependent threshold statistic, so it can differ substantially across basins and systems and should not be confused with a fixed sparsity budget. The top-k support S_{top8} selects the eight largest-magnitude latent coordinates. It is the deployment-like object because every state receives a route key without a magnitude threshold. The relative-threshold support S_{rel} marks a coordinate active when its magnitude is at least 10% of the largest latent magnitude for that state. It removes global scale effects and was useful for centered local-law diagnostics, but it is too fragmented as an exact support router and is therefore retained only as a sensitivity object. Support families are produced by greedy Jaccard merging at threshold 0.5. In our results, S_{abs} saturates the conditional entropy $H(B \mid S_{\text{abs}})$ on the deep slice for all sparse rows and is therefore not a discriminator on its own; $H(S_{\text{abs}} \mid B)$, U_{exact}, and $|S_{\text{abs}}|$ are the discriminative columns there. S_{top8} is the only support definition for which the dense MLP control's failure on routing cannot be explained by route-availability artifacts.

E Multibasin benchmark inventory

Table 8 lists the 17 multibasin systems used in this paper, their basin counts available at evaluation, and the main-text display item that uses each system.

F Per-seed distributions

Table 1 reports cross-system summaries of per-system IQM-over-seeds. The corresponding per-(system, seed) distributions are shown below. Figure 4 shows the paper-facing strip plot for the matched boundary-emphasized rows. Figure 5 plots per-(system, seed) histograms of forecasting MSE at every paper-facing horizon. Figure 6 plots per-(system, seed) histograms of $H(B \mid S_{\text{abs}})$, $H(S_{\text{abs}} \mid B)$, and U_{exact} on the deep-state slice. Figure 7 plots the same per-seed distributions for the Dysts $dt\times30$ benchmark. The distributions are heavy-tailed in log-MSE; we therefore report the cross-system $\log_{10}$-IQR alongside the IQM in table 1. The cross-system $\log_{10}$-IQR is ≤ 1.97 for every row at every horizon (the highest values come from the rows whose worst-system seeds blow up at $H1000$, dominated by `multiwell_strong_transition`), and the per-system rank ordering of rows is stable across horizons.

Table 8: The 17 multibasin benchmark systems. Geometry types: *multiwell* = multistable potential systems; *gated* = gated local-linear systems with explicit transitions between regions; *polygonal* = arrangement of point attractors at vertices of a polygon; *checkerboard* = grid-based potential; *cascade* = depth-cascade variants. Basin counts are revealed only at evaluation.

System	Geometry	Basins	Used in
`multiwell_strong_transition`	multiwell	4	Tab. 1, Fig. 4
`gated_local_linear`	gated	4	Tab. 1, Tab. 2
`gated_transfer_linear`	gated	4	Tab. 1, Tab. 3
`arrested_spiral`	polygonal	4	Tab. 2
`cal_asymmetric_3`	polygonal	3	Tab. 1
`cal_high_cross_3`	polygonal	3	Tab. 1
`cal_hexagon_6`	polygonal	6	Tab. 1
`cal_octagon_8`	polygonal	8	Tab. 1
`cal_pentagon_5`	polygonal	5	Tab. 1
`cal_square_4`	polygonal	4	Tab. 1
`checkerboard_potential`	checkerboard	9	Tab. 1
`duffing_triple_well`	multiwell	3	Tab. 1, Fig. 4
`snic_multi`	polygonal	3	Tab. 1
`transition_routes_4`	gated	4	Tab. 1
`var_depth_gradient_4`	cascade	4	Tab. 1
`var_diamond_4`	polygonal	4	Tab. 1
`var_l_shape_5`	polygonal	5	Tab. 1

G Full per-system tables

Tables 9–11 render the per-(system, candidate) effect sizes and Holm-corrected Wilcoxon p-values that underlie the K/N counts in table 1 and the forest plot in figure 8. Each cell gives the per-system median paired $\log_{10}$-MSE difference (candidate minus matched-sampling Dense MLP, no-shrink baseline) over the common completed seeds, up to 15 per system, and the corresponding Holm-corrected one-sided paired Wilcoxon p-value (Holm correction across the 17 systems within each horizon-and-candidate column). Bold entries indicate Holm-corrected $p < 0.05$ in favor of the candidate. Systems are sorted by the median across candidates of the per-system effect; the same ordering is used in all three tables for visual cross-reference.

The raw per-(system, seed) values, per-system bootstrap 95% CIs of the paired median, and the corresponding paired t-test p-values are released in the supplementary materials. The supplementary CSVs include per-system records, the row-level K/N summary, and the cross-system table used in the main text; the manifest lists the names and the source forecasting CSVs.

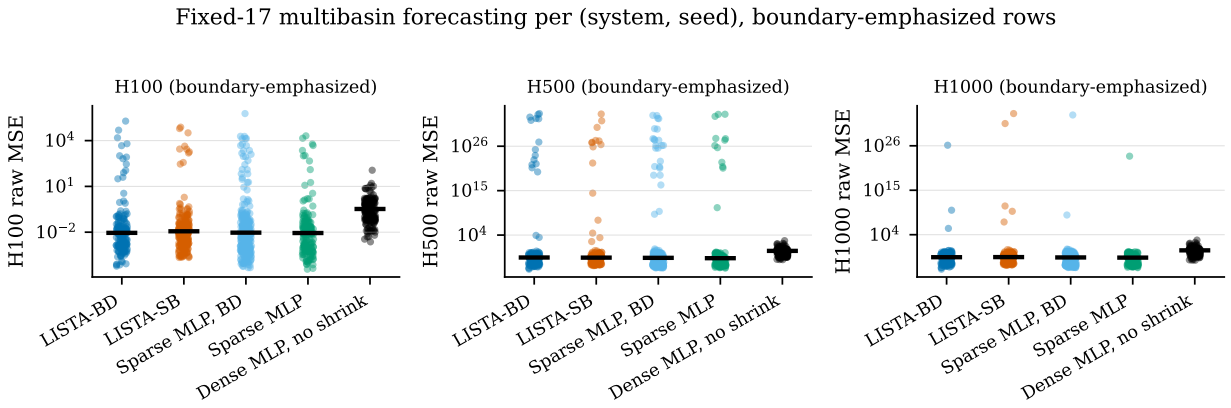


Figure 4: Per-(system, seed) forecasting MSE at $H100$, $H500$, and $H1000$ on the multibasin benchmark. Black bars are IQM values. The Dense MLP has a much heavier upper tail at every horizon.

Per-system audit of the interpretability columns

Why the wrong-support denominator is 13 and not 17. The wrong-support ablation builds, for each system, a dictionary mapping each basin to its canonical support mask – the most-common exact support among the deep-slice states of that basin. The ablated rollout draws a mask from a basin different from the state's own basin and holds that wrong mask fixed, so the test requires at least two distinct deep-slice canonical masks. Four of the 17 benchmark systems – `arrested_spiral` (5 basins), `cal_asymmetric_3` (3), `duffing_triple_well` (3), `var_l_shape_5` (5) – have multiple basins by design but a deep slice (top quartile by basin-depth margin) concentrated in a single basin: one basin's attractor has a much larger margin to the second-nearest centroid than the others, so the global top-quartile filter selects only that basin's states. This holds across all 7 trained model rows and all 9 support-threshold definitions, confirming it is a property of the system's depth-margin geometry rather than of any encoder. The wrong-support ablation is undefined on those four systems by construction, and the corresponding K/N denominator in table 1 and tables 14 and 15 is 13 instead of 17. The four omitted systems still appear in the per-system tables with "––" entries. A queued robustness re-evaluation under a per-basin top-quartile depth criterion – which admits each basin's relative-deep states and is expected to restore 17/17 coverage on these four systems – is documented in `docs/PER_BASIN_DEEP_SLICE_PLAN.md`; its results will appear here as a separate appendix table once the re-eval completes, and the global-slice numbers in table 1 and the per-system tables below are the source-of-truth in the meantime.

Tables 12–15 render the full per-(system, candidate) breakdown of the four interpretability columns of table 1. Each cell gives the per-system median paired difference between the candidate and the matched-sampling Dense MLP no-shrink baseline (over the common completed seeds, up to 15 per system), and the corresponding Holm-corrected one-sided paired Wilcoxon p-value (Holm correction across the 17 systems within each candidate column). Bold entries indicate Holm-corrected $p < 0.05$ in favor of the candidate. The alternative direction is "candidate < baseline" for $|S_{\text{abs}}|$ and $H(B \mid F_{\text{abs}})$ (smaller is better) and "candidate > baseline" for the wrong-support ablation ratios (larger is better because the support is more functionally meaningful when the wrong one hurts more).

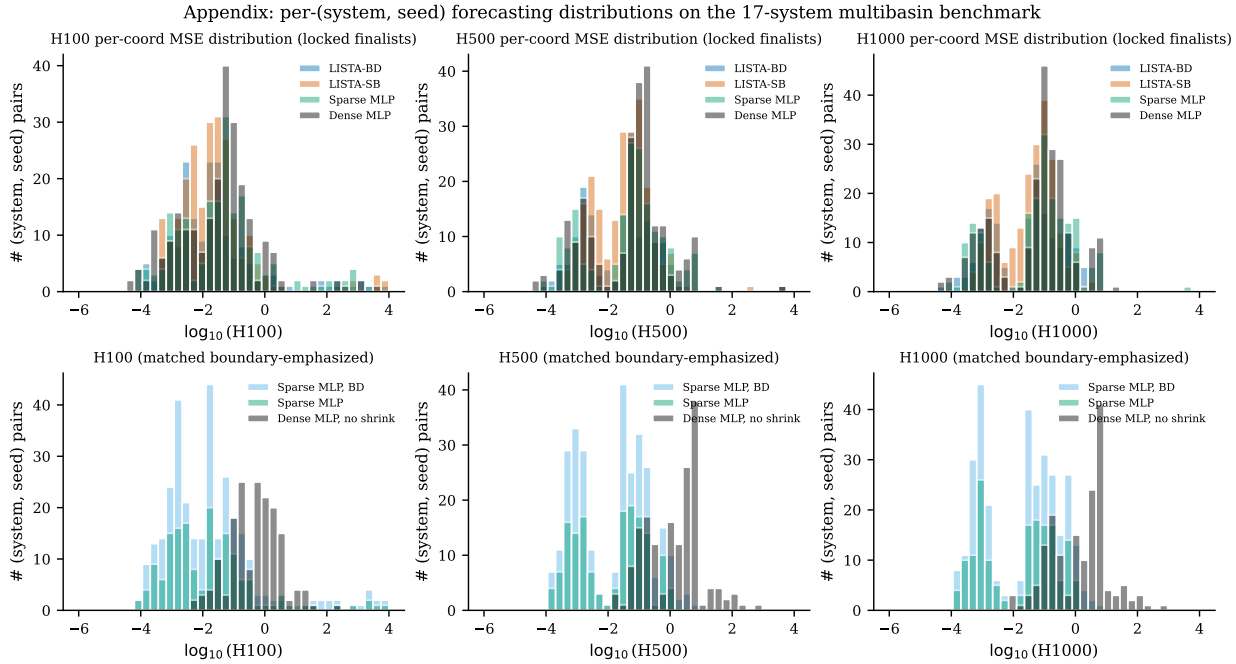


Figure 5: Per-(system, seed) forecasting MSE distributions on the multibasin benchmark. Each row is a horizon ($H100$, $H500$, $H1000$); the top row shows the LISTA finalists with the standard-sampling MLP controls (for completeness), and the bottom row shows the matched boundary-emphasized MLP controls used in the main-text comparison. Histograms are over the $\log_{10}$ raw MSE.

H Support-routing robustness

Table 16 reports the deep-state counterpart to the all-state routing table in the main text. The deep slice removes boundary-adjacent states, so it is useful as a robustness check rather than the primary deployment setting.

I Full per-system Dysts tables

The per-(system, seed) Dysts MSE values are released in the collected `forecasting_rows.csv` artifacts. Table 4 reports cross-system IQMs after first summarizing seeds within each system. The $dt\times30$ packet is fully finite in median coverage at every reported horizon, but the per-seed distributions remain heavy-tailed on several systems, so the main table and figures 3b and 7 should be read together with the within-system $[K/12]$ counts rather than as aggregate-only evidence. The older tiny-step $H\leq60000$ packet is useful as a stress-test diagnostic because it exposed divergent block-diagonal LISTA rollouts, but it is not pooled with the coarser-step table.

J Falsification: oracle, random, and latent-cluster controls

The routing experiments include two falsification controls that we summarise here. (i) Oracle-conditioned A_{basin} fits beat the global K on 93–100% of evaluated deep rows for every model family, including the dense no-sparsity MLP. This says local centred laws exist for many representations once a good partition is supplied; the sparse-latent contribution is that the model itself supplies an

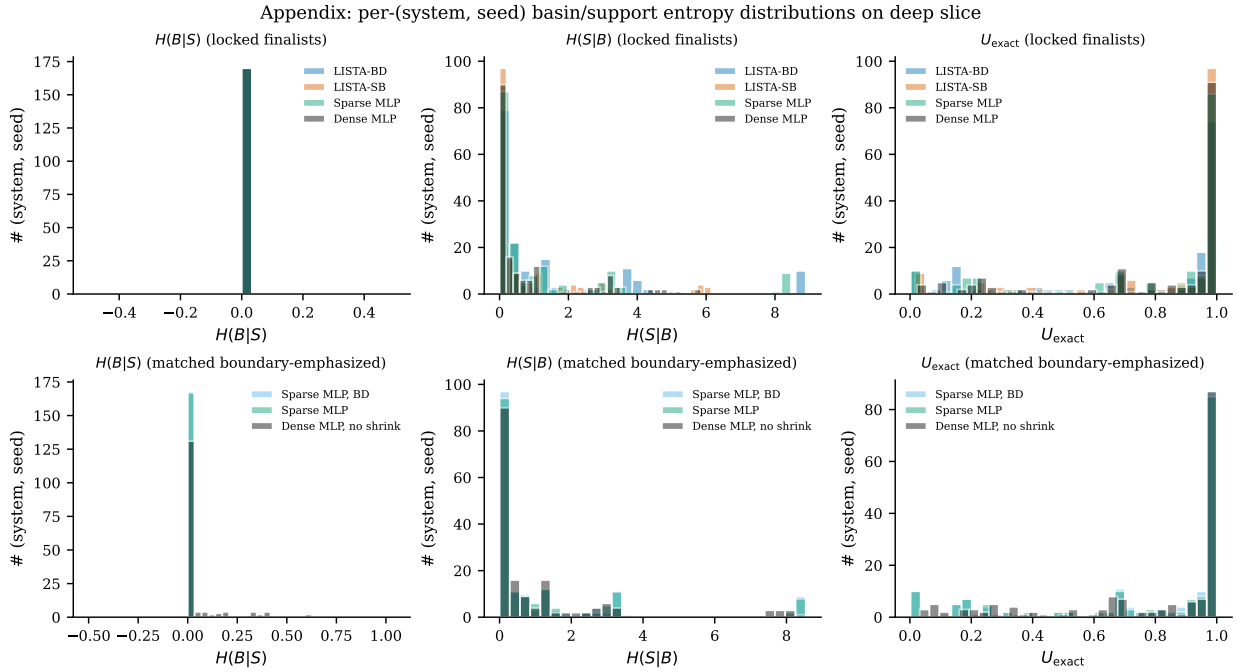


Figure 6: Per-(system, seed) support–basin diagnostic distributions on the deep-state slice. Histograms show $H(B \mid S_{\text{abs}})$, $H(S_{\text{abs}} \mid B)$, and exact-support uniqueness for the rows summarized in table 1.

inspectable, label-free partition that does this work. (ii) Random count-matched partitions and latent k-means clusters do not reproduce the routed/global ratios in table 16: support-conditioned routing improves on these controls by an order of magnitude on the deep slice for the sparse-latent rows.

References

Omri Azencot, N. Benjamin Erichson, Vanessa Lin, and Michael Mahoney. Forecasting Sequential Data Using Consistent Koopman Autoencoders. In *Proceedings of the 37th International Conference on Machine Learning*, pages 475–485. PMLR, November 2020.

Steven L. Brunton, Bingni W. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Koopman Invariant Subspaces and Finite Linear Representations of Nonlinear Dynamical Systems for Control. *PLOS ONE*, 11(2), February 2016. ISSN 1932-6203. doi: 10.1371/journal.pone.0150171. URL https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0150171.

Steven L. Brunton, Marko Budišić, Eurika Kaiser, and J. Nathan Kutz. Modern Koopman Theory for Dynamical Systems. *SIAM Review*, 64(2):229–340, May 2022. ISSN 0036-1445. doi: 10.1137/21M1401243. URL https://doi.org/10.1137/21M1401243.

Scott Shaobing Chen, David L. Donoho, and Michael A. Saunders. Atomic Decomposition by Basis Pursuit. *SIAM Journal on Scientific Computing*, 20(1):33–61, 1998. doi: 10.1137/S1064827596304010. URL https://doi.org/10.1137/S1064827596304010.

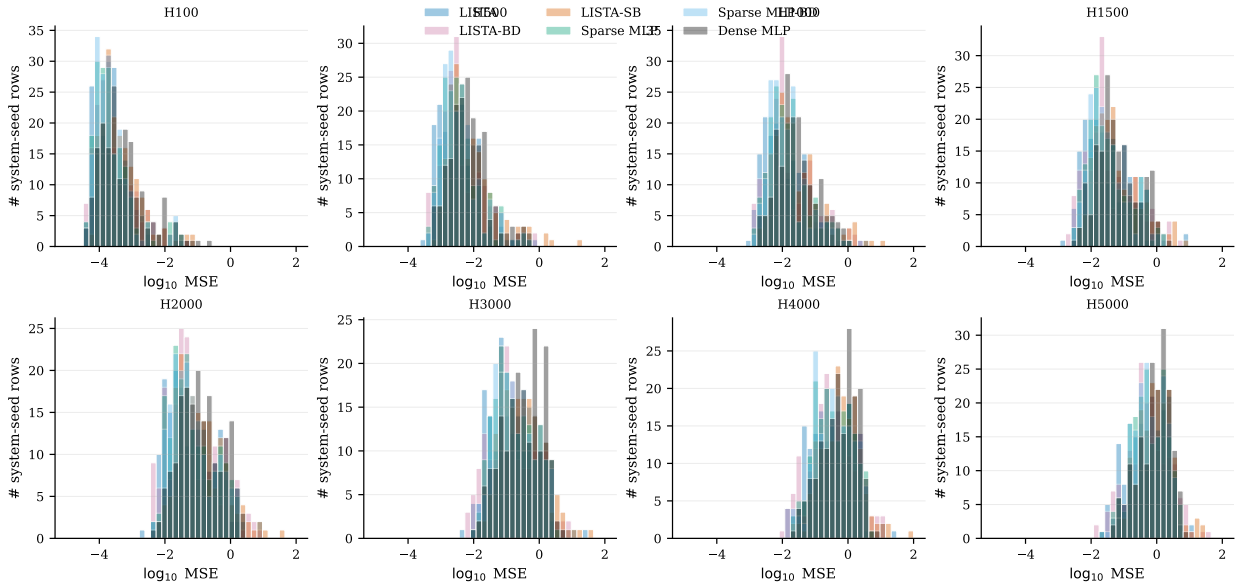


Figure 7: Per-(system, seed) forecasting MSE distributions on the Dysts $dt\times30$ benchmark.

David L. Donoho and Michael Elad. Optimally sparse representation in general (nonorthogonal) dictionaries via ℓ^1 minimization. *Proceedings of the National Academy of Sciences*, 100(5): 2197–2202, March 2003. doi: 10.1073/pnas.0437847100. URL https://doi.org/10.1073/pnas.0437847100.

Mahan Fathi, Clement Gehring, Jonathan Pilault, David Kanaa, Pierre-Luc Bacon, and Ross Goroshin. Course correcting koopman representations. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=A18gWgc5mi.

William Gilpin. Chaos as an interpretable benchmark for forecasting and data-driven modelling. In *Advances in Neural Information Processing Systems (NeurIPS) 2021*, 2021. URL http://arxiv.org/abs/2110.05266.

William Gilpin. Model scale versus domain knowledge in statistical forecasting of chaotic systems. *Physical Review Research*, 5(4):043252, December 2023. doi: 10.1103/PhysRevResearch.5.043252. URL https://link.aps.org/doi/10.1103/PhysRevResearch.5.043252.

Karol Gregor and Yann LeCun. Learning Fast Approximations of Sparse Coding. In *Proceedings of the 27th International Conference on Machine Learning*, ICML'10, pages 399–406, Haifa, Israel, 2010. Omnipress. ISBN 978-1-60558-907-7. doi: 10.5555/3104322.3104374. URL https://dl.acm.org/doi/abs/10.5555/3104322.3104374.

D. M. Grobman. Homeomorphism of systems of differential equations. *Doklady Akademii Nauk SSSR*, 128:880–881, 1959.

Lars Grüne and Jürgen Pannek. *Nonlinear Model Predictive Control*. Communications and Control Engineering. Springer International Publishing, Cham, 2017. ISBN 978-3-319-46023-9 978-3-319-46024-6. doi: 10.1007/978-3-319-46024-6. URL http://link.springer.com/10.1007/978-3-319-46024-6.

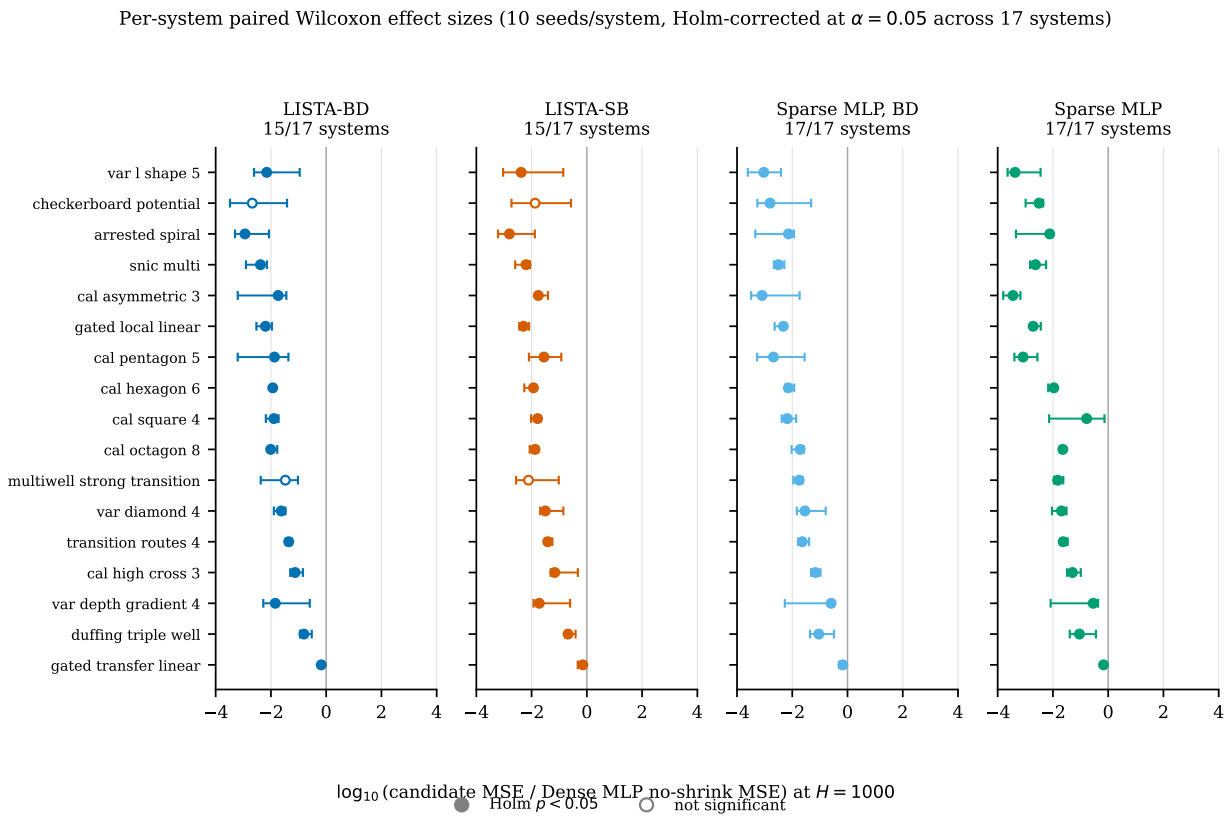


Figure 8: Per-system effect sizes at H=1000 versus the Dense MLP baseline. Each point is one benchmark system; the x-coordinate is the per-system median paired $\log_{10}$-MSE difference (candidate minus baseline) over the common completed seeds, up to 15 per system, with whiskers showing the 95% bootstrap interval. Filled markers clear Holm-corrected α=0.05. Numerical values are in table 11.

Philip Hartman. A lemma in the theory of structural stability of differential equations. *Proceedings of the American Mathematical Society*, 11(4):610–620, 1960. doi: 10.1090/S0002-9939-1960-0121542-7.

R.E. Kalman. On the general theory of control systems. *1st International IFAC Congress on Automatic and Remote Control, Moscow, USSR, 1960*, 1(1):491–502, August 1960. ISSN 1474-6670. doi: 10.1016/S1474-6670(17)70094-8. URL https://www.sciencedirect.com/science/article/pii/S1474667017700948.

B. O. Koopman. Hamiltonian Systems and Transformation in Hilbert Space. *Proceedings of the National Academy of Sciences*, 17(5):315–318, May 1931. doi: 10.1073/pnas.17.5.315. URL https://www.pnas.org/doi/10.1073/pnas.17.5.315.

B. O. Koopman and J. v. Neumann. Dynamical Systems of Continuous Spectra. *Proceedings of the National Academy of Sciences*, 18(3):255–263, March 1932. doi: 10.1073/pnas.18.3.255. URL https://www.pnas.org/doi/abs/10.1073/pnas.18.3.255.

Milan Korda and Igor Mezić. Linear predictors for nonlinear dynamical systems: Koopman operator

Table 9: Per-system effect sizes and Holm-corrected Wilcoxon p-values at $H = 100$ on the 17-system multibasin benchmark, against the matched-sampling Dense MLP, no-shrink baseline. Δ is the per-system median paired $\log_{10}$-MSE difference (candidate − baseline) over the common completed seeds, up to 15 per system; p_{H} is the one-sided paired Wilcoxon signed-rank p-value, Holm-corrected across the 17 systems within each candidate column. Bold Δ: Holm-corrected $p < 0.05$ in favor of the candidate.

System	LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
var_l_shape_5	**−1.94**	$<10^{-3}$	**−1.90**	$<10^{-3}$	**−2.05**	$<10^{-3}$	**−2.23**	$<10^{-3}$
checkerboard_potential	−0.45	0.229	+0.72	1.000	+1.02	1.000	+0.12	1.000
arrested_spiral	**−2.09**	$<10^{-3}$	**−2.03**	$<10^{-3}$	**−1.93**	$<10^{-3}$	**−1.90**	$<10^{-3}$
snic_multi	**−2.03**	$<10^{-3}$	**−1.91**	$<10^{-3}$	**−2.09**	$<10^{-3}$	**−2.13**	$<10^{-3}$
cal_asymmetric_3	**−1.92**	$<10^{-3}$	**−1.75**	$<10^{-3}$	**−2.00**	$<10^{-3}$	**−2.44**	$<10^{-3}$
gated_local_linear	**−2.19**	$<10^{-3}$	**−2.20**	$<10^{-3}$	**−2.35**	$<10^{-3}$	**−2.52**	$<10^{-3}$
cal_pentagon_5	**−1.58**	$<10^{-3}$	**−1.20**	$<10^{-3}$	**−1.89**	$<10^{-3}$	**−2.16**	$<10^{-3}$
cal_hexagon_6	**−1.73**	$<10^{-3}$	**−1.85**	$<10^{-3}$	**−1.96**	$<10^{-3}$	**−1.83**	$<10^{-3}$
cal_square_4	**−1.62**	$<10^{-3}$	**−1.58**	$<10^{-3}$	**−1.39**	$<10^{-3}$	**−1.02**	$<10^{-3}$
cal_octagon_8	**−1.90**	$<10^{-3}$	**−1.80**	$<10^{-3}$	**−1.68**	$<10^{-3}$	**−1.63**	$<10^{-3}$
multiwell_strong_transition	+2.39	1.000	+2.16	1.000	+2.34	1.000	+2.25	1.000
var_diamond_4	**−1.80**	$<10^{-3}$	**−1.84**	$<10^{-3}$	**−1.46**	$<10^{-3}$	**−1.67**	$<10^{-3}$
transition_routes_4	**−1.77**	$<10^{-3}$	**−1.90**	$<10^{-3}$	**−2.36**	$<10^{-3}$	**−2.47**	$<10^{-3}$
cal_high_cross_3	**−1.12**	$<10^{-3}$	**−1.19**	$<10^{-3}$	**−1.10**	$<10^{-3}$	**−1.17**	$<10^{-3}$
var_depth_gradient_4	**−1.33**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.05**	$<10^{-3}$	**−0.82**	$<10^{-3}$
duffing_triple_well	**−0.51**	0.019	**−0.35**	0.013	**−0.81**	0.007	**−0.78**	0.002
gated_transfer_linear	**−0.21**	0.017	**−0.31**	0.013	**−0.36**	$<10^{-3}$	**−0.27**	0.013
K/17 *Holm* $p < 0.05$	K=15 / N=17		K=15 / N=17		K=15 / N=17		K=15 / N=17	

meets model predictive control. *Automatica*, 93:149–160, July 2018. ISSN 00051098. doi: 10.1016/j.automatica.2018.03.046. URL http://arxiv.org/abs/1611.03537.

Matthew D. Kvalheim and Shai Revzen. Existence and uniqueness of global Koopman eigenfunctions for stable fixed points and periodic orbits. *Physica D: Nonlinear Phenomena*, 425:132959, November 2021. ISSN 0167-2789. doi: 10.1016/j.physd.2021.132959. URL https://www.sciencedirect.com/science/article/pii/S0167278921001160.

Yueheng Lan and Igor Mezić. Linearization in the large of nonlinear systems and Koopman operator spectrum. *Physica D: Nonlinear Phenomena*, 242(1):42–53, January 2013. ISSN 0167-2789. doi: 10.1016/j.physd.2012.08.017.

Scott Linderman, Matthew Johnson, Andrew Miller, Ryan Adams, David Blei, and Liam Paninski. Bayesian Learning and Inference in Recurrent Switching Linear Dynamical Systems. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, pages 914–922. PMLR, April 2017. URL https://proceedings.mlr.press/v54/linderman17a.html.

Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton. Deep learning for universal linear

Table 10: Per-system effect sizes and Holm-corrected Wilcoxon p-values at $H = 500$ on the 17-system multibasin benchmark. Format identical to table 9.

System	LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
var_l_shape_5	**−2.04**	$<10^{-3}$	**−2.03**	$<10^{-3}$	**−2.73**	$<10^{-3}$	**−2.96**	$<10^{-3}$
checkerboard_potential	+7.30	1.000	+6.89	1.000	+1.90	1.000	+0.15	0.330
arrested_spiral	**−2.57**	$<10^{-3}$	**−2.57**	$<10^{-3}$	**−2.40**	$<10^{-3}$	**−2.33**	$<10^{-3}$
snic_multi	**−2.45**	$<10^{-3}$	**−2.31**	$<10^{-3}$	**−2.44**	$<10^{-3}$	**−2.49**	$<10^{-3}$
cal_asymmetric_3	**−2.12**	$<10^{-3}$	**−1.94**	$<10^{-3}$	**−2.53**	$<10^{-3}$	**−3.15**	$<10^{-3}$
gated_local_linear	**−2.24**	$<10^{-3}$	**−2.27**	$<10^{-3}$	**−2.29**	$<10^{-3}$	**−2.52**	$<10^{-3}$
cal_pentagon_5	**−2.10**	$<10^{-3}$	**−1.61**	$<10^{-3}$	**−2.42**	$<10^{-3}$	**−2.77**	$<10^{-3}$
cal_hexagon_6	**−2.01**	$<10^{-3}$	**−2.18**	$<10^{-3}$	**−2.32**	$<10^{-3}$	**−2.16**	$<10^{-3}$
cal_square_4	**−1.89**	$<10^{-3}$	**−1.81**	$<10^{-3}$	**−1.67**	0.001	**−1.17**	$<10^{-3}$
cal_octagon_8	**−2.09**	$<10^{-3}$	**−1.91**	$<10^{-3}$	**−1.81**	$<10^{-3}$	**−1.74**	$<10^{-3}$
multiwell_strong_transition	+25.40	1.000	+23.56	1.000	+25.03	1.000	+24.89	1.000
var_diamond_4	**−1.58**	$<10^{-3}$	**−1.40**	$<10^{-3}$	**−1.41**	$<10^{-3}$	**−1.75**	$<10^{-3}$
transition_routes_4	**−1.52**	$<10^{-3}$	**−1.49**	$<10^{-3}$	**−1.68**	$<10^{-3}$	**−1.86**	$<10^{-3}$
cal_high_cross_3	**−0.97**	$<10^{-3}$	**−1.03**	$<10^{-3}$	**−1.17**	$<10^{-3}$	**−1.26**	$<10^{-3}$
var_depth_gradient_4	**−1.45**	$<10^{-3}$	**−1.47**	$<10^{-3}$	**−1.21**	$<10^{-3}$	**−0.92**	$<10^{-3}$
duffing_triple_well	**−0.81**	$<10^{-3}$	**−0.65**	$<10^{-3}$	**−1.07**	0.001	**−1.05**	$<10^{-3}$
gated_transfer_linear	**+1.19**	0.023	**−0.21**	$<10^{-3}$	**−0.18**	0.001	**−0.18**	$<10^{-3}$
$K/17$ *Holm* $p < 0.05$	K=15 / N=17		K=15 / N=17		K=15 / N=17		K=15 / N=17	

embeddings of nonlinear dynamics. *Nature Communications*, 9(1):4950, November 2018. ISSN 2041-1723. doi: 10.1038/s41467-018-07210-0.

Giorgos Mamakoukas, Maria Castano, Xiaobo Tan, and Todd Murphey. Local Koopman Operators for Data-Driven Control of Robotic Systems. In *Robotics: Science and Systems XV*. Robotics: Science and Systems Foundation, June 2019. ISBN 978-0-9923747-5-4. doi: 10.15607/RSS.2019.XV.054. URL http://www.roboticsproceedings.org/rss15/p54.pdf.

D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, June 2000. ISSN 0005-1098. doi: 10.1016/S0005-1098(99)00214-9. URL https://www.sciencedirect.com/science/article/pii/S0005109899002149.

Igor Mezić. Spectral properties of dynamical systems, model reduction and decompositions. *Nonlinear Dynamics*, 41(1–3):309–325, 2005. doi: 10.1007/s11071-005-2824-x.

Bruno A. Olshausen and David J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381(6583):607–609, June 1996. ISSN 1476-4687. doi: 10.1038/381607a0. URL https://doi.org/10.1038/381607a0.

Samuel E. Otto and Clarence W. Rowley. Linearly recurrent autoencoder networks for learning dynamics. *SIAM Journal on Applied Dynamical Systems*, 18(1):558–593, 2019. doi: 10.1137/18M1177846.

Table 11: Per-system effect sizes and Holm-corrected Wilcoxon p-values at $H = 1000$ on the 17-system multibasin benchmark. This is the numerical version of the forest plot in figure 8. Format identical to table 9.

System	LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
var_l_shape_5	$\mathbf{-2.15}$	$<10^{-3}$	$\mathbf{-2.38}$	$<10^{-3}$	$\mathbf{-3.03}$	$<10^{-3}$	$\mathbf{-3.37}$	$<10^{-3}$
checkerboard_potential	-2.68	0.083	-1.87	0.381	$\mathbf{-2.81}$	0.013	$\mathbf{-2.50}$	0.008
arrested_spiral	$\mathbf{-2.94}$	$<10^{-3}$	$\mathbf{-2.80}$	$<10^{-3}$	$\mathbf{-2.14}$	$<10^{-3}$	$\mathbf{-2.12}$	$<10^{-3}$
snic_multi	$\mathbf{-2.38}$	$<10^{-3}$	$\mathbf{-2.20}$	$<10^{-3}$	$\mathbf{-2.50}$	$<10^{-3}$	$\mathbf{-2.63}$	$<10^{-3}$
cal_asymmetric_3	$\mathbf{-1.74}$	$<10^{-3}$	$\mathbf{-1.76}$	$<10^{-3}$	$\mathbf{-3.09}$	$<10^{-3}$	$\mathbf{-3.45}$	$<10^{-3}$
gated_local_linear	$\mathbf{-2.20}$	$<10^{-3}$	$\mathbf{-2.29}$	$<10^{-3}$	$\mathbf{-2.32}$	$<10^{-3}$	$\mathbf{-2.72}$	$<10^{-3}$
cal_pentagon_5	$\mathbf{-1.87}$	$<10^{-3}$	$\mathbf{-1.55}$	$<10^{-3}$	$\mathbf{-2.68}$	$<10^{-3}$	$\mathbf{-3.08}$	$<10^{-3}$
cal_hexagon_6	$\mathbf{-1.93}$	$<10^{-3}$	$\mathbf{-1.93}$	$<10^{-3}$	$\mathbf{-2.14}$	$<10^{-3}$	$\mathbf{-1.97}$	$<10^{-3}$
cal_square_4	$\mathbf{-1.89}$	$<10^{-3}$	$\mathbf{-1.78}$	$<10^{-3}$	$\mathbf{-2.18}$	0.001	$\mathbf{-0.78}$	$<10^{-3}$
cal_octagon_8	$\mathbf{-2.01}$	$<10^{-3}$	$\mathbf{-1.87}$	$<10^{-3}$	$\mathbf{-1.72}$	$<10^{-3}$	$\mathbf{-1.64}$	$<10^{-3}$
multiwell_strong_transition	-1.48	0.083	-2.11	0.083	$\mathbf{-1.75}$	0.008	$\mathbf{-1.83}$	0.008
var_diamond_4	$\mathbf{-1.63}$	$<10^{-3}$	$\mathbf{-1.50}$	$<10^{-3}$	$\mathbf{-1.54}$	$<10^{-3}$	$\mathbf{-1.69}$	$<10^{-3}$
transition_routes_4	$\mathbf{-1.36}$	$<10^{-3}$	$\mathbf{-1.42}$	$<10^{-3}$	$\mathbf{-1.64}$	$<10^{-3}$	$\mathbf{-1.63}$	$<10^{-3}$
cal_high_cross_3	$\mathbf{-1.13}$	$<10^{-3}$	$\mathbf{-1.16}$	$<10^{-3}$	$\mathbf{-1.16}$	$<10^{-3}$	$\mathbf{-1.29}$	$<10^{-3}$
var_depth_gradient_4	$\mathbf{-1.84}$	$<10^{-3}$	$\mathbf{-1.72}$	$<10^{-3}$	$\mathbf{-0.59}$	$<10^{-3}$	$\mathbf{-0.54}$	$<10^{-3}$
duffing_triple_well	$\mathbf{-0.80}$	$<10^{-3}$	$\mathbf{-0.68}$	$<10^{-3}$	$\mathbf{-1.04}$	0.001	$\mathbf{-1.04}$	$<10^{-3}$
gated_transfer_linear	$\mathbf{-0.18}$	$<10^{-3}$	$\mathbf{-0.14}$	0.001	$\mathbf{-0.16}$	0.002	$\mathbf{-0.17}$	$<10^{-3}$
$K/17$ *Holm* $p < 0.05$	K=15 / N=17		K=15 / N=17		K=17 / N=17		K=17 / N=17	

Shaowu Pan and Karthik Duraisamy. On the lifting and reconstruction of nonlinear systems with multiple invariant sets. *Nonlinear Dynamics*, 112(12):10157–10165, June 2024. ISSN 1573-269X. doi: 10.1007/s11071-024-09581-0. URL https://doi.org/10.1007/s11071-024-09581-0.

Clarence W. Rowley, Igor Mezić, Shervin Bagheri, Philipp Schlatter, and Dan S. Henningson. Spectral analysis of nonlinear flows. *Journal of Fluid Mechanics*, 641:115–127, 2009. doi: 10.1017/S0022112009992059.

Peter J. Schmid. Dynamic mode decomposition of numerical and experimental data. *Journal of Fluid Mechanics*, 656:5–28, 2010. ISSN 0022-1120. doi: 10.1017/S0022112010001217.

Haojie Shi and Max Q.-H. Meng. Deep Koopman Operator With Control for Nonlinear Systems. *IEEE Robotics and Automation Letters*, 7(3):7700–7707, July 2022. ISSN 2377-3766. doi: 10.1109/LRA.2022.3184036.

Naoya Takeishi, Yoshinobu Kawahara, and Takehisa Yairi. Learning koopman invariant subspaces for dynamic mode decomposition. In *Advances in Neural Information Processing Systems 30*, volume 30, 2017. URL https://papers.nips.cc/paper/6713-learning-koopman-invariant-subspaces-for-dynamic-mode-decomposition.

F.D. Torrisi and A. Bemporad. Hysdel-a tool for generating computational hybrid models for

Table 12: Per-system effect sizes and Holm-corrected Wilcoxon p-values for $|S_{\text{abs}}|$, the mean active-coordinate count under the absolute-threshold support on the deep slice, on the 17-system multibasin benchmark. Alternative: candidate $|S_{\text{abs}}| <$ baseline $|S_{\text{abs}}|$. Lower (more negative) is better for the candidate. Latent dimension is d_z=64 for LISTA-SB and d_z=256 for LISTA-BD and the MLP rows.

System	LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
snic_multi	**−174.21**	$<10^{-3}$	**−171.07**	$<10^{-3}$	**−170.02**	$<10^{-3}$	**−169.93**	$<10^{-3}$
gated_local_linear	**−167.67**	$<10^{-3}$	**−167.29**	$<10^{-3}$	**−159.44**	$<10^{-3}$	**−159.44**	$<10^{-3}$
cal_asymmetric_3	**−158.00**	$<10^{-3}$	**−156.12**	$<10^{-3}$	**−163.00**	$<10^{-3}$	**−162.97**	$<10^{-3}$
transition_routes_4	**−162.38**	$<10^{-3}$	**−163.99**	$<10^{-3}$	**−152.85**	$<10^{-3}$	**−152.73**	$<10^{-3}$
gated_transfer_linear	**−152.76**	$<10^{-3}$	**−155.40**	$<10^{-3}$	**−158.88**	$<10^{-3}$	**−158.43**	$<10^{-3}$
duffing_triple_well	**−163.31**	$<10^{-3}$	**−163.67**	$<10^{-3}$	**−148.02**	$<10^{-3}$	**−148.80**	$<10^{-3}$
arrested_spiral	**−151.91**	$<10^{-3}$	**−153.06**	$<10^{-3}$	**−154.19**	$<10^{-3}$	**−154.19**	$<10^{-3}$
var_diamond_4	**−144.83**	$<10^{-3}$	**−157.41**	$<10^{-3}$	**−149.83**	$<10^{-3}$	**−149.50**	$<10^{-3}$
cal_pentagon_5	**−156.55**	$<10^{-3}$	**−157.12**	$<10^{-3}$	**−141.89**	$<10^{-3}$	**−142.43**	$<10^{-3}$
cal_hexagon_6	**−155.01**	$<10^{-3}$	**−156.11**	$<10^{-3}$	**−142.86**	$<10^{-3}$	**−142.86**	$<10^{-3}$
var_l_shape_5	**−151.90**	$<10^{-3}$	**−155.03**	$<10^{-3}$	**−142.74**	$<10^{-3}$	**−142.74**	$<10^{-3}$
cal_square_4	**−152.11**	$<10^{-3}$	**−153.07**	$<10^{-3}$	**−140.53**	$<10^{-3}$	**−140.78**	$<10^{-3}$
cal_octagon_8	**−147.55**	$<10^{-3}$	**−152.64**	$<10^{-3}$	**−143.20**	$<10^{-3}$	**−143.34**	$<10^{-3}$
multiwell_strong_transition	**−140.80**	$<10^{-3}$	**−132.42**	$<10^{-3}$	**−149.52**	$<10^{-3}$	**−149.52**	$<10^{-3}$
var_depth_gradient_4	**−149.53**	$<10^{-3}$	**−152.15**	$<10^{-3}$	**−140.31**	$<10^{-3}$	**−140.15**	$<10^{-3}$
checkerboard_potential	**−144.59**	$<10^{-3}$	**−144.64**	$<10^{-3}$	**−138.22**	$<10^{-3}$	**−138.22**	$<10^{-3}$
cal_high_cross_3	**−143.99**	$<10^{-3}$	**−146.03**	$<10^{-3}$	**−127.04**	$<10^{-3}$	**−133.00**	$<10^{-3}$
K/17 *Holm* $p < 0.05$	K=17 / N=17		K=17 / N=17		K=17 / N=17		K=17 / N=17	

analysis and synthesis problems. *IEEE Transactions on Control Systems Technology*, 12(2): 235–249, 2004. doi: 10.1109/TCST.2004.824309.

Jonathan H. Tu, Clarence W. Rowley, Dirk M. Luchtenburg, Steven L. Brunton, and J. Nathan Kutz. On dynamic mode decomposition: Theory and applications. *Journal of Computational Dynamics*, 1(2):391–421, December 2014. ISSN 2158-2491. doi: 10.3934/jcd.2014.1.391. URL https://www.aimsciences.org/en/article/doi/10.3934/jcd.2014.1.391.

Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A Data–Driven Approximation of the Koopman Operator: Extending Dynamic Mode Decomposition. *Journal of Nonlinear Science*, 25(6):1307–1346, December 2015. ISSN 1432-1467. doi: 10.1007/s00332-015-9258-5. URL https://doi.org/10.1007/s00332-015-9258-5.

Matthew O. Williams, Clarence W. Rowley, and Ioannis G. Kevrekidis. A kernel-based method for data-driven koopman spectral analysis. *Journal of Computational Dynamics*, 2(2):247–265, May 2016. ISSN 2158-2491. doi: 10.3934/jcd.2015005. URL https://www.aimsciences.org/article/doi/10.3934/jcd.2015005.

Table 13: Per-system effect sizes and Holm-corrected Wilcoxon p-values for $H(B \mid F_{\text{abs}})$ (basin uncertainty given the support *family* built from S_{abs}, deep slice). Alternative: candidate $H(B \mid F_{\text{abs}}) <$ baseline. Lower is better for the candidate.

System	LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
transition_routes_4	**−1.38**	$<10^{-3}$	**−1.38**	$<10^{-3}$	**−1.38**	$<10^{-3}$	**−1.38**	$<10^{-3}$
cal_square_4	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$
var_diamond_4	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$	**−1.37**	$<10^{-3}$
multiwell_strong_transition	**−1.30**	$<10^{-3}$	**−1.22**	$<10^{-3}$	**−1.28**	$<10^{-3}$	**−1.29**	$<10^{-3}$
cal_octagon_8	**−1.33**	$<10^{-3}$	**−1.36**	$<10^{-3}$	**−1.09**	0.001	**−1.09**	0.001
snic_multi	**−1.09**	$<10^{-3}$	**−1.09**	$<10^{-3}$	**−1.09**	$<10^{-3}$	**−1.09**	$<10^{-3}$
gated_local_linear	**−1.07**	$<10^{-3}$	**−1.07**	$<10^{-3}$	**−1.07**	$<10^{-3}$	**−1.07**	$<10^{-3}$
gated_transfer_linear	**−1.06**	$<10^{-3}$	**−1.06**	$<10^{-3}$	**−1.06**	$<10^{-3}$	**−1.06**	$<10^{-3}$
cal_hexagon_6	**−0.92**	$<10^{-3}$	**−0.82**	$<10^{-3}$	**−0.66**	0.001	**−0.66**	0.001
cal_pentagon_5	**−0.64**	$<10^{-3}$	**−0.64**	$<10^{-3}$	**−0.64**	0.002	**−0.64**	0.001
var_depth_gradient_4	**−0.62**	$<10^{-3}$	**−0.62**	$<10^{-3}$	**−0.62**	$<10^{-3}$	**−0.62**	$<10^{-3}$
checkerboard_potential	**−1.08**	$<10^{-3}$	**−1.21**	$<10^{-3}$	**−0.07**	0.001	**−0.08**	0.001
cal_high_cross_3	**−0.01**	$<10^{-3}$	**−0.01**	$<10^{-3}$	**−0.00**	0.007	**−0.01**	0.001
cal_asymmetric_3	+0.00	–	+0.00	–	+0.00	–	+0.00	–
duffing_triple_well	+0.00	–	+0.00	–	+0.00	1.000	+0.00	–
arrested_spiral	+0.00	–	+0.00	–	+0.00	–	+0.00	–
var_l_shape_5	+0.00	–	+0.00	–	+0.00	–	+0.00	–
K/17 *Holm* $p < 0.05$	K=13 / N=13		K=13 / N=13		K=13 / N=14		K=13 / N=13	

Table 14: Per-system effect sizes and Holm-corrected Wilcoxon p-values for the wrong-support-over-base MSE ratio at h=1 (deep slice). Alternative: candidate ratio > baseline. Higher is better for the candidate (using the wrong support hurts the prediction more, indicating the model genuinely uses the support).

System	LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
multiwell_strong_transition	**+506.16**	$<10^{-3}$	**+463.88**	$<10^{-3}$	**+1346.70**	$<10^{-3}$	**+1346.70**	$<10^{-3}$
checkerboard_potential	**+1165.01**	$<10^{-3}$	**+1038.03**	$<10^{-3}$	**+2847.10**	$<10^{-3}$	**+2611.92**	$<10^{-3}$
var_depth_gradient_4	**+4566.67**	$<10^{-3}$	**+8591.21**	$<10^{-3}$	**+1036.02**	$<10^{-3}$	**+1036.02**	$<10^{-3}$
cal_pentagon_5	**+5222.11**	$<10^{-3}$	**+4328.40**	$<10^{-3}$	**+1359.93**	$<10^{-3}$	**+1231.67**	$<10^{-3}$
cal_high_cross_3	**+1471.93**	$<10^{-3}$	**+6293.08**	$<10^{-3}$	**+2932.96**	$<10^{-3}$	**+3155.16**	$<10^{-3}$
cal_octagon_8	**+6473.72**	$<10^{-3}$	**+8701.56**	$<10^{-3}$	**+3832.77**	$<10^{-3}$	**+3782.78**	$<10^{-3}$
gated_transfer_linear	**+4362.02**	$<10^{-3}$	**+5438.15**	$<10^{-3}$	**+9156.48**	$<10^{-3}$	**+7648.11**	$<10^{-3}$
cal_square_4	**+13062.48**	$<10^{-3}$	**+17067.95**	$<10^{-3}$	**+5820.10**	$<10^{-3}$	**+5395.17**	$<10^{-3}$
var_diamond_4	**+5461.40**	$<10^{-3}$	**+16175.88**	$<10^{-3}$	**+8872.06**	$<10^{-3}$	**+10718.65**	$<10^{-3}$
cal_hexagon_6	**+7892.75**	$<10^{-3}$	**+9778.76**	$<10^{-3}$	**+12232.14**	$<10^{-3}$	**+12232.14**	$<10^{-3}$
transition_routes_4	**+110074.20**	$<10^{-3}$	**+76351.70**	$<10^{-3}$	**+32116.22**	$<10^{-3}$	**+27463.47**	$<10^{-3}$
snic_multi	**+18483.93**	$<10^{-3}$	**+23485.14**	$<10^{-3}$	**+209357.97**	$<10^{-3}$	**+322696.68**	$<10^{-3}$
gated_local_linear	**+55127.58**	$<10^{-3}$	**+112929.91**	$<10^{-3}$	**+281342.90**	$<10^{-3}$	**+362761.28**	$<10^{-3}$
K/17 *Holm* $p < 0.05$	K=13 / N=13		K=13 / N=13		K=13 / N=13		K=13 / N=13	

Table 15: Per-system effect sizes and Holm-corrected Wilcoxon p-values for the wrong-support-over-base MSE ratio at h=20. Alternative: candidate > baseline. Higher is better.

System	LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}	Δ	p_{H}
multiwell_strong_transition	−0.22	1.000	−0.61	1.000	−0.78	0.982	−0.75	0.968
cal_high_cross_3	−0.34	1.000	+0.79	0.188	+3.46	0.073	**+13.92**	0.015
cal_octagon_8	**+4.45**	$<10^{-3}$	**+2.99**	0.001	**+6.57**	$<10^{-3}$	**+8.59**	$<10^{-3}$
var_depth_gradient_4	**+0.92**	0.027	**+3.01**	0.001	**+13.60**	$<10^{-3}$	**+13.60**	$<10^{-3}$
cal_square_4	**+9.39**	$<10^{-3}$	**+3.43**	$<10^{-3}$	**+10.59**	$<10^{-3}$	**+10.56**	$<10^{-3}$
cal_pentagon_5	**+12.30**	$<10^{-3}$	**+7.70**	0.002	**+11.04**	$<10^{-3}$	**+8.97**	$<10^{-3}$
checkerboard_potential	**+3.35**	0.001	**+2.68**	0.001	**+21.28**	$<10^{-3}$	**+30.18**	$<10^{-3}$
gated_transfer_linear	**+22.31**	0.002	**+14.53**	0.001	**+27.51**	$<10^{-3}$	**+22.12**	$<10^{-3}$
cal_hexagon_6	**+25.99**	$<10^{-3}$	**+7.23**	$<10^{-3}$	**+45.83**	$<10^{-3}$	**+45.83**	$<10^{-3}$
var_diamond_4	**+1.75**	0.005	**+40.92**	$<10^{-3}$	**+49.36**	$<10^{-3}$	**+49.23**	$<10^{-3}$
transition_routes_4	**+176.48**	$<10^{-3}$	**+214.10**	$<10^{-3}$	**+208.70**	$<10^{-3}$	**+154.08**	$<10^{-3}$
snic_multi	**+281.74**	$<10^{-3}$	**+185.64**	$<10^{-3}$	**+573.45**	$<10^{-3}$	**+625.93**	$<10^{-3}$
gated_local_linear	**+362.21**	$<10^{-3}$	**+1169.28**	$<10^{-3}$	**+2948.75**	$<10^{-3}$	**+4073.56**	$<10^{-3}$
K/17 *Holm* $p < 0.05$	K=11 / N=13		K=11 / N=13		K=11 / N=13		K=12 / N=13	

Table 16: Deep-state support-routed forecasting with top-8 routes. Entries are routed/global MSE-ratio IQMs with within-system Wilcoxon/Holm counts in brackets. LISTA-SB uses the matched d_z=256 control; lower is better.

H100

Model	Gated	Support-local	Family-local
LISTA-SB	0.883 [1/17]	0.875 [2/17]	0.0791 [13/17]
LISTA-BD	0.922 [1/17]	0.911 [2/17]	**0.0324** [13/17]
Sparse MLP, BD	**0.829** [0/17]	0.803 [0/17]	0.0910 [5/17]
Sparse MLP	0.847 [0/17]	**0.792** [1/17]	0.661 [4/17]
Dense MLP	0.991 [0/17]	0.989 [0/17]	662 [0/17]

H1000

Model	Gated	Support-local	Family-local
LISTA-SB	0.850 [8/17]	0.838 [7/17]	0.00109 [15/17]
LISTA-BD	**0.733** [7/17]	**0.756** [8/17]	$\mathbf{9.34\times10^{-5}}$ [15/17]
Sparse MLP, BD	0.991 [1/17]	0.962 [1/17]	0.233 [16/17]
Sparse MLP	0.871 [1/17]	0.810 [1/17]	119 [16/17]
Dense MLP	0.903 [0/17]	0.972 [0/17]	533 [1/17]